

Project 2: Surgical Tool Sorting using EB100

ENPM662: Introduction To Robot Modeling

Anirudh Swarankar (UID: 121150653)

Munyaradzi P Antony (UID: 120482731)

Pranav Deshakulkarni Manjunath (UID: 121162090)

Varad Nerlekar (UID: 120501135)



Under the guidance of Dr. Reza Monfaredi

Contents

1	Introduction	4
2	Application	5
2.1	Automation of Tool Handling	5
2.2	Process Workflow for Tool Picking and Sterilization	5
3	Robot Type	6
3.1	Articulated Robotic Arm	6
3.2	Structural Design	7
3.3	Key Features Enabling Tool Picking	7
3.4	Degrees of Freedom and Kinematics	7
3.5	End-Effector Versatility	7
4	DOFs and Dimensions	8
4.1	Degrees of Freedom (DOF)	8
4.2	Dimensions	8
4.3	Advantages of the Configuration	8
4.4	Specifications Summary	9
5	CAD Models	10
5.1	CAD file format conversions	10
5.2	Deployment to Ignition Gazebo	10
5.3	Challenges and Mitigation	10
6	D-H Parameters	11
6.1	Frame Assignment	11
6.2	Denavit-Hartenberg Representation	12
7	Forward Kinematics	13
7.1	Transformation Matrices	13
8	Forward Kinematics Validation	14
8.1	Verification with Peter Corke's Robotics Toolbox for MATLAB	16
9	Inverse Kinematics	17
10	Inverse Kinematics Validation	17
11	Velocity Kinematics – EB100	17
11.1	Calculating the 0Z_n of the Transformations	17
12	Workspace Study	21
12.1	Denavit-Hartenberg (DH) Parameters	21
12.2	Workspace Reachability Calculation	21
12.3	Challenges and Observations	22
12.4	Conclusion	22
13	Assumptions	23
14	Control Method	24
14.1	Feedback Control Loop	24
14.2	Dynamic Modeling and Control	24
14.3	Integration with ROS	24
14.4	Challenges and Solutions	24
15	Perception Module	25

15.1 Tool Detection Process	25
15.2 Data Integration and Pose Estimation	26
15.3 Cross-Referencing with Task Order	26
16 Gazebo & RViz Visualization	26
16.1 Gazebo Plugins	26
16.1.1 EB100 Manipulator Simulation	26
16.1.2 Key Components	27
16.1.3 Challenges and Mitigations	27
16.2 RViz Visualization	28
17 Problems Faced	28
17.1 Singularities in 6DOF Robot Arm	28
17.1.1 Addressing Singularities	28
17.2 Bottlenecks and Mitigations in Simulation	28
17.3 Bottlenecks and Mitigations in Modeling	29
17.4 Bottlenecks and Mitigations in Software Implementation	30
18 Lessons Learned	30
18.1 Key Take Aways	30
18.2 Conclusion	31
19 Conclusions	31
20 Future Work	32
21 Links to GitHub Repository and demonstration videos	33
References	33
22 Appendix	34
Appendix	34
A Project Components	34
B Workflow	35

1 Introduction

The healthcare sector faces significant challenges in automating the sorting, handling, and sterilization of surgical instruments. These tasks are labor-intensive, error-prone, and critical to patient safety. Manual sorting can lead to inefficiencies, delays, and safety risks, including potential infections. Automation in this domain can improve operational efficiency, reduce costs, and enhance safety by streamlining workflows and minimizing human error.

The primary motivations for automation in surgical instrument handling include:

- **Enhanced Surgical Efficiency:** Automate tool sorting, ensuring precise organization and quick accessibility of surgical tools, reducing the time spent by medical staff in tool retrieval during procedures.
- **Minimized Human Error:** Reduce the risk of human error in misplacing or misidentifying instruments, contributing to a safer surgical environment.
- **Sterility Maintenance:** Maintain hygiene and safety standards by minimizing human contact with tools during operations in sterile environments.
- **Integration with Smart Systems:** Enable real-time tracking of tools through integration with surgical management systems, ensuring accurate inventory management and reducing the risk of tool shortages or mismatches.
- **Application in High-Volume Facilities:** Handle the high volume of tools required for various procedures in large hospitals or surgical centers, enabling faster preparation and turnover between surgeries.

Several studies have explored robotic solutions for automating surgical instrument handling:

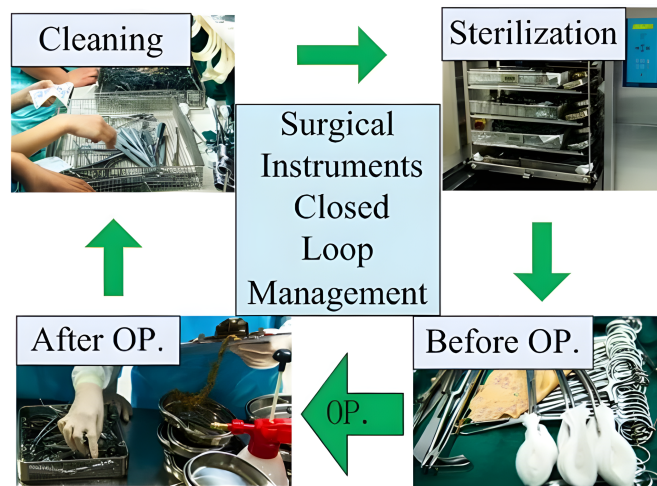


Figure 1: Closed Loop Management of Surgical Tools

1. **Fast Surgical Instruments Identification System:** A deep learning-based system [1] was developed to assist robots in identifying and picking surgical instruments. Using image processing, the system can quickly recognize diverse and similar instruments through automated segmentation and bounding box labeling. By leveraging publicly available datasets, it achieves high identification accuracy and reduced manual effort, though challenges remain in handling densely packed instruments.
2. **Surgical Instruments Sorting System with Stereo Vision:** This system [2] uses stereo vision combined with impedance control and adaptive robot speed to improve sorting precision. Despite achieving high recognition accuracy, the system struggles with identifying instruments in oblique orientations, pointing to the need for further enhancements in recognition and adaptability.
3. **Robotic Handling of Instruments in Cluttered Trays:** A vision-guided robotic system [3] addresses the challenge of handling instruments in cluttered trays. The system uses a 4-DOF pose estimation algorithm and a compliant electromagnetic gripper to handle instruments precisely. While the system demonstrates high success rates, further work is needed to optimize error handling and sequence planning.

2 Application

The using a robotic manipulator such as a Robot arm attached with a relevant gripper offers a transformative solution for automating surgical tool sterilization processes. It offers precise motion control, compact design, and versatile functionality make it an ideal candidate for improving efficiency and hygiene in medical environments. This section focuses on its application in picking surgical tools for sterilization.

2.1 Automation of Tool Handling

Surgical tool sterilization is a critical process in healthcare, requiring meticulous handling to prevent contamination and ensure patient safety. Traditionally, this task is performed manually, introducing risks such as human error, inefficiency, and exposure to biohazards. The robotic manipulator addresses these challenges by automating the picking, sorting, and placement of tools into sterilization trays.

2.2 Process Workflow for Tool Picking and Sterilization

The workflow for using a robot arm in surgical tool sterilization involves several automated steps:

1. **Managing orders for dispensing tools:** The system receives and processes orders for required surgical tools, ensuring that the correct items are selected based on the procedure and surgeon's specifications. It efficiently prioritizes tool availability and tracks inventory in real-time to minimize delays.
2. **Tool Detection and Identification:** Vision systems and pre-trained detection algorithms, such as YoloV8 and MaskRCNN, identify surgical tools on cluttered trays. The perception leverages synthetic datasets generated from CAD models to recognize tools accurately.
3. **Grasp Planning and Execution:** The manipulator computes optimal grasp points for each tool based on its dimensions and position. Inverse kinematics algorithms calculate the necessary joint angles to reach and grasp the tool without colliding with adjacent objects.
4. **Placement into Sterilization Trays:** Once a tool is grasped, the manipulator transports it to a designated sterilization tray, ensuring organized placement for efficient sterilization. Tools are grouped based on size, type, or usage requirements.
5. **Integration with Smart Systems:** The system can be connected to a network of smart devices and cloud-based management platforms, allowing seamless tracking of tool usage, sterilization status, and order completion. This enables real-time updates and data sharing across the hospital's inventory management systems and operating room workflows.

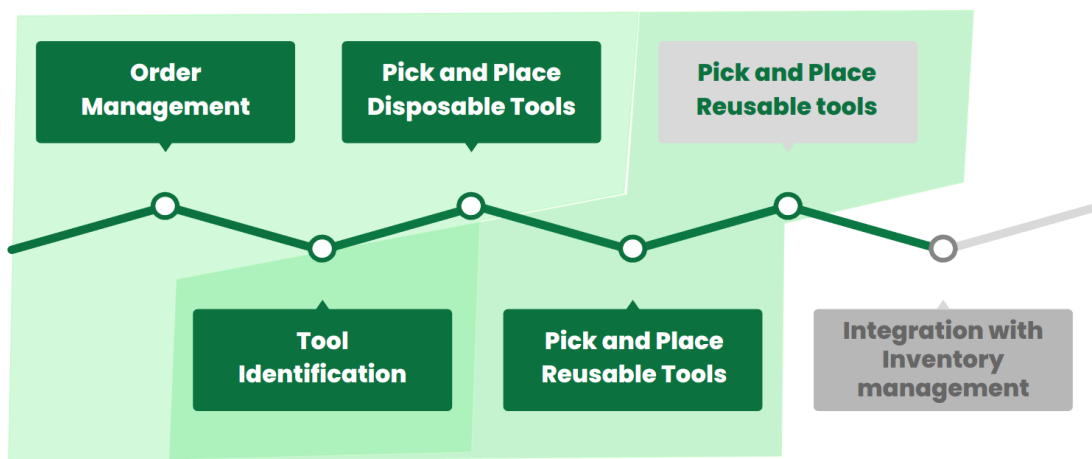


Figure 2: Solution Workflow

3 Robot Type

The EB100 is a six-degree-of-freedom (DOF) robotic arm designed for precision tasks in industrial and medical environments. As a lightweight and compact manipulator, it combines high flexibility with reliable performance, making it suitable for applications such as surgical tool handling, pick-and-place tasks, and automation in constrained spaces.

We chose this robot to overcome unique challenges of sorting surgical tools, including safely and reliably handling tools with complex geometries. Its versatile and modular design allows it to adapt to various operational demands.

- **Sufficient Degrees of Freedom (DOF):** The EB100 is a 6-DOF robot, which is the minimum necessary to achieve full pose control (position and orientation) in a 3D workspace. In robotic manipulation, achieving an accurate **pose** (the position of an object in space) is critical. However, to also control the **orientation** (the way an object is positioned, such as its rotation), a robot needs at least 6 DOF. While a 5-DOF robot would suffice for positioning an object, it would not have the required flexibility to orient it precisely.

The 7-DOF robots, while offering additional **redundancy** (extra degrees of freedom that help with task flexibility and avoiding obstacles), were deemed unnecessary for our project. The EB100's 6 DOF offers the perfect balance, giving us full control over both position and orientation while avoiding the complexity and computational overhead of working with a 7-DOF or higher system.

- **Flexibility in Task Space:** The 6-DOF configuration of the EB100 ensures that the robot can reach any desired point in its workspace with a specific orientation, which is crucial for precise manipulation of objects such as surgical tools. This flexibility is important for our task, where tools have complex geometries, and precision handling is necessary. For example, tools might need to be manipulated or picked from different angles, and a 6-DOF robot allows us to perform these tasks efficiently within a constrained workspace.
- **Availability of STL Files for Hardware Implementation:** A significant factor in the selection of the EB100 was the **Availability of STL files** for its design: A significant factor in the selection of the EB100 was the availability of the STL files on Thingiverse. These files allow us to implement the robot's hardware easily for physical prototyping and testing. By utilizing the existing 3D models, we were able to quickly design and 3D print components necessary for the robot's integration with the task environment. This was a key advantage, as having readily available files shortened the development time and allowed for easier customization and adaptation of the robot to our specific needs.
- **Compact and Lightweight Design:** The EB100's compact size and lightweight nature make it well-suited for working in constrained environments, such as those often found in medical and industrial settings. Its small footprint also makes it easy to integrate into a robotic system with limited space, such as our tool handling and sorting task, where it needs to operate in close proximity to the tray of surgical tools.
- **Reliability and Precision:** The EB100 is known for its high accuracy and repeatability, which are essential in tasks where precise movements and tool handling are required, such as in surgery or manufacturing automation. The high reliability of the robot ensures that it will perform consistently under the varying conditions of the task.
- **Modular Design:** The modular nature of the EB100 robot allows for adaptability in different operational scenarios. Its structure can be easily modified or extended for various types of tasks, making it a versatile solution for our needs. Whether performing basic pick-and-place actions or more complex tool handling, the EB100 can be reconfigured to meet the specific demands of the task.

3.1 Articulated Robotic Arm

The EB100 is classified as an articulated robotic arm. This type of manipulator is characterized by its multiple rotary joints, which provide it with high dexterity and a wide range of motion. The arm consists of six revolute joints, enabling full spatial control of its end-effector. These joints allow the robot to achieve complex poses, including rotations and translations in three-dimensional space. This capability makes the EB100 ideal for tasks requiring precise positioning and orientation, such as grasping surgical tools or assembling small components.

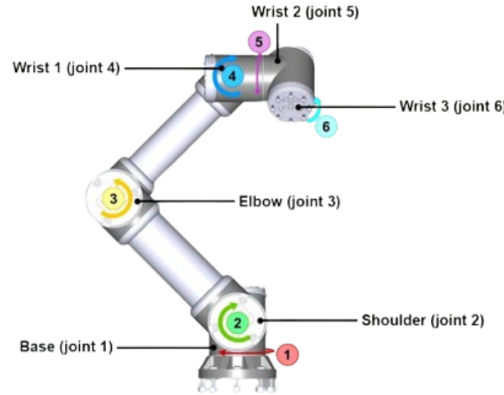


Figure 3: Robot Type

3.2 Structural Design

The EB100 features a compact and lightweight design with a total weight of approximately 10 kg. Its small footprint allows it to operate effectively in confined workspaces, such as medical sterilization rooms or assembly lines. Despite its compact structure, the EB100 boasts a robust build capable of handling payloads of up to 1 kg. This balance of strength and size ensures that the manipulator can perform a wide range of tasks without compromising accuracy or efficiency.

3.3 Key Features Enabling Tool Picking

The EB100 manipulator's design includes several features that enhance its ability to handle surgical tools:

- **Precision and Accuracy:** The six degrees of freedom (DOF) of the EB100 allow for precise positioning and orientation of the end-effector. This ensures the accurate grasping of tools, even in cluttered or confined spaces.
- **Advanced Gripper Design:** The manipulator's gripper is equipped with adaptive constraints to securely hold tools of various shapes and sizes. The gripper's force is calibrated to prevent damage to delicate instruments while maintaining a firm hold.
- **High-Resolution Sensors:** Integrated vision systems and force sensors enable the EB100 to identify and interact with tools accurately. Vision systems facilitate the detection of tool positions, while force sensors provide feedback to ensure secure handling.
- **Compact Workspace:** The EB100 has a compact footprint and a 600 mm workspace radius, allowing it to operate in sterilization rooms with limited space. Its design ensures seamless integration into existing workflows.

3.4 Degrees of Freedom and Kinematics

The robot's six degrees of freedom provide flexibility in motion, enabling it to reach any point within its workspace with precise control over its orientation. Its joints are driven by high-torque actuators and monitored by encoders, ensuring smooth and stable movements. The EB100 employs kinematic modeling techniques, such as Denavit-Hartenberg (DH) parameterization, to calculate joint angles and end-effector positions. This enables seamless control and integration into various automation workflows.

3.5 End-Effector Versatility

The EB100 is designed to accommodate a range of end-effectors, including grippers, suction cups, and specialized tools for different applications. For surgical tool handling, its gripper is optimized to securely grasp instruments of varying shapes and sizes. The gripper's adaptive design and precise force control make it suitable for delicate operations.

4 DOFs and Dimensions

The EB100 manipulator is a versatile 6-degree-of-freedom (DOF) robotic arm designed for precise and adaptable operations in a variety of environments. Its compact design and carefully engineered structure make it a highly suitable choice for tasks like surgical tool sorting, as demonstrated in this project.

4.1 Degrees of Freedom (DOF)

The EB100 manipulator features six degrees of freedom, enabling full spatial manipulation of its end-effector. These six joints include three for positioning the arm in space (X, Y, Z axes) and three for orienting the end-effector (roll, pitch, yaw). This configuration provides the manipulator with sufficient flexibility to perform complex tasks, such as reaching constrained areas, orienting objects at precise angles, and performing delicate pick-and-place operations.

The manipulator's joints are designed with high-resolution encoders, enabling precise joint angle measurements and feedback control. Each joint's movement is driven by robust actuators that provide smooth and stable motion. The six-DOF configuration ensures the manipulator can achieve a full range of poses, essential for applications requiring fine-grained adjustments and multi-axis motion.



Figure 4: EB100 Details and Dimensions

4.2 Dimensions

The EB100 manipulator has a compact footprint and a workspace radius of 600 mm, making it ideal for operations in confined spaces, such as medical environments or assembly lines. Despite its small size, the arm offers a significant payload capacity of up to 1 kg, allowing it to handle a wide range of objects, from small surgical tools to lightweight industrial components.

The arm's base width and height are designed to ensure stability during operation, even when the arm is extended to its maximum reach. The arm's segments are proportioned to balance maneuverability and structural strength, enabling precise movements without compromising robustness.

4.3 Advantages of the Configuration

The six-DOF structure and compact dimensions of the EB100 provide several advantages:

- **Flexibility:** The six degrees of freedom allow the manipulator to operate in environments with complex geometries.
- **Precision:** High-resolution encoders and efficient actuators enable accurate positioning and orientation of the end-effector.

- **Space Efficiency:** The 600 mm workspace radius and compact base design make the EB100 suitable for operations in limited spaces.
- **Payload Handling:** The arm can manage loads of up to 1 kg, meeting the requirements of various applications, including delicate surgical tool handling.

In summary, the EB100 manipulator's 6-DOF design and optimized dimensions enable it to perform precise, flexible, and reliable operations, making it an invaluable tool for both simulated and real-world applications.

4.4 Specifications Summary

The following specifications outline the primary functional and design requirements for the robotic arm and gripper system:

1. **Degrees of Freedom:** The Robot arm is a 6 *DOF* spatial serial manipulator to enable complex movements for tool handling.
2. **Joints:** The *EB100* robot arm is a 6*R* serial manipulator, and has 6 revolute joints.
3. **Reach:** The *EB100* arm provides adequate reach to access multiple tool boxes and sorting areas within the workspace.
4. **Payload Capacity:** Sufficient to handle a range of surgical tools, from lightweight instruments to moderately heavy items.
5. **Sensors:** The robot will use a high-resolution RGB camera to locate, identify, categorize and sort the tools based on visual features such as shape, size and color.
6. **Gripper Design:** The EBG5-G1 2*D* gripper with a precision grip, designed specifically for delicate surgical tools, ensuring secure handling without damage.
7. **Control System:** The system will use a *PID* Controller for joint position control and tool manipulation, and *ROS2* to simulate arm movement, gripper control, and task sequencing.
8. **Accuracy and Repeatability:** High positional accuracy is essential for precise tool handling, with repeatability to ensure consistent sorting operations across cycles.

These specifications are designed to meet the rigorous standards required in surgical environments, where accuracy, reliability, and safety are paramount. Through iterative testing and validation, the robotic arm and gripper system aims to achieve these specifications, ensuring readiness for real-world application in operating room settings.

5 CAD Models

The creation and deployment of CAD files for the EB100 manipulator were pivotal steps in ensuring an accurate and functional simulation. Autodesk Fusion 360 was employed as the primary tool for modifying the available STL files to STEP files and then imported to SolidWorks for creating an assembly, taking advantage of its robust 3D modeling capabilities and seamless integration with downstream simulation platforms.

5.1 CAD file format conversions

The CAD files available from ToolBox Robotics, the manufacturer of EB100 and EBG20 Gripper, provided the STL files for the robot on open source platforms. Due to the lack of SolidWorks capabilities with STL, we had to use Autodesk Fusion 360 to convert the STL files to STEP files. Fusion 360's parametric modeling capabilities enabled iterative refinements, such as adjusting joint radii, arm lengths, and end-effector configurations to optimize performance, which helped dealing with conversion issues and scaling.

The converted STEP files were then imported into SolidWorks to be converted to SLDPRT files and then created an assembly. We then had to ensure the origins, coordinate frames and axes were defined correctly as the file conversion led to some issues. In the assembly, the robot arm and the gripper were mated together in its home configuration and then a URDF package was generated using the SW2URDF plugin.

5.2 Deployment to Ignition Gazebo

Once the CAD model was finalized, it was exported as STL files for integration into the Ignition Gazebo simulation environment. The STL files were converted into Unified Robot Description Format (URDF) using SolidWorks URDF exporter tools. This step involved defining the kinematic and dynamic properties of the manipulator, such as joint limits, collision geometries, and inertial attributes. Accurate collision models and inertia values were assigned to each component to ensure realistic physical interactions during simulation.

Ignition Gazebo was chosen over the classic Gazebo environment as the simulation platform for its enhanced capabilities and modern architecture. Unlike the older Gazebo, Ignition Gazebo is designed to provide a more modular and flexible framework, with improved performance, better support for distributed simulation, and an overall more intuitive interface for developers. The ability to integrate with advanced tools like ROS 2 and its high scalability were key factors in this decision. Additionally, Ignition Gazebo's advanced rendering and physics engines offered a more realistic simulation of the robot's interaction with its environment. By using Ignition Gazebo, we were able to simulate complex scenarios more effectively, validate the robot's performance with greater precision, and ensure the robot's movements were accurately represented in the virtual environment, including during pick-and-place tasks and trajectory following.

5.3 Challenges and Mitigation

- **URDF Exporter Limitations:** The SolidWorks URDF exporter was prone to crashes and inconsistencies, leading to delays in deployment. To mitigate this, progress was saved incrementally, and manual corrections were made to collision and inertial attributes to ensure accuracy.
- **Translation Artifacts:** While the CAD models appeared flawless in Fusion 360, artifacts such as misaligned joints and broken assemblies arose during the Gazebo integration. Simplifying the model by merging components and removing unnecessary features reduced these issues.
- **STL File Limitations:** The STL files lacked interaction data, such as collision properties and inertial attributes, requiring manual updates to the URDF file. Collision tags were added and verified to enhance the simulation's fidelity.
- **Iterative Refinement:** Small misalignments in joint orientations caused errors in motion during simulation. These were corrected through iterative testing and adjustments to the URDF file.

6 D-H Parameters

Denavit-Hartenberg (DH) parameters are a systematic method to represent the kinematic chains of robotic arms. They simplify the mathematical modeling of robots by providing a standard notation to describe the relative positions and orientations of adjacent links. The four DH parameters—link length, link twist, link offset, and joint angle—allow for the precise description of each joint in terms of a common coordinate system, making it easier to derive the kinematic equations necessary for controlling the robot's movement.[4]

DH method was used to develop the kinematic model of the EB100 robot in this work because of its versatility and acceptability for modeling of any number of joints and links of a serial manipulator regardless of complexity.

In the below sections frame assignment for EB100 is presented as defined by DH table.

6.1 Frame Assignment

The below Figure 17 depicts the Frame assignment for the EB100 attached to an EBG-20 end effector required by the DH convention.

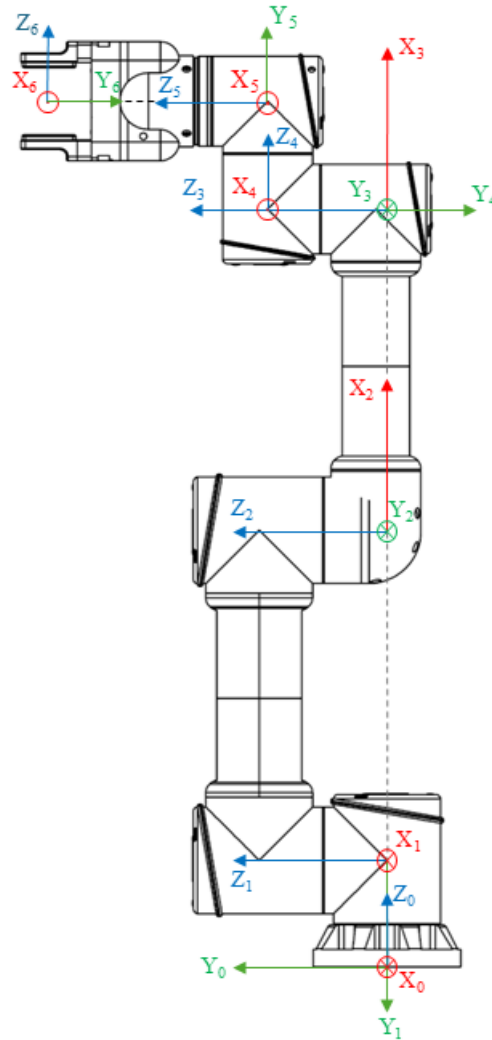


Figure 5: EB100 Frame assignment

6.2 Denavit-Hartenberg Representation

Based on the above frames assigned for DH, we determine the DH parameters for the 6 joints as follows:

i	$a_i(m)$	$\alpha_i(rad)$	$d_i(m)$	$\theta_i(rad)$
1	0	$-\pi/2$	$d_1 = 0.076$	θ_1
2	$a_2 = 0.235$	0	0	$\theta_2 - \pi/2$
3	$a_3 = 0.230$	0	0	θ_3
4	0	$-\pi/2$	$d_4 = 0.085$	$\theta_4 - \pi/2$
5	0	$\pi/2$	$d_5 = 0.076$	θ_5
6	0	$-\pi/2$	$d_6 = 0.15878$	θ_6

Table 1: EB100 Denavit–Hartenberg parameters

7 Forward Kinematics

Forward kinematics refers to the use of the kinematic equations of a robot to compute the position of the end-effector from specified values for the joint parameters. (Source Wiki) Given a kinematic chain composed of links and joints with multiple degree of freedom, finding the position and orientation of the end-effector in the operational workspace when all the joint parameters are known. The Forward Kinematics problem translate to the following statement.

$$\text{Find } T_0^n(\theta) \text{ given } \theta_i \in [1, \dots, n]$$

where T_0^n is Homogeneous Transformation Matrix between the frame 0 located at the base joint of the robot and the final frame n, attached at the End- Effector.

7.1 Transformation Matrices

Using DH table transformation matrix between n and n-1 frame can be written as, Matrix T_{n-1}^n :

$$\begin{bmatrix} \cos(\theta_i) & -\sin(\theta_i) \cos(\alpha_i) & \sin(\alpha_i) \sin(\theta_i) & a_i \cos(\theta_i) \\ \sin(\theta_i) & \cos(\alpha_i) \cos(\theta_i) & -\sin(\alpha_i) \cos(\theta_i) & a_i \sin(\theta_i) \\ 0 & \sin(\alpha_i) & \cos(\alpha_i) & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Decomposition

$$T_1^{Base} = \begin{bmatrix} \cos(\theta_1) & 0 & \sin(\theta_1) & 0 \\ \sin(\theta_1) & 0 & -\cos(\theta_1) & 0 \\ 0 & -1 & 0 & 0.076 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_2^1 = \begin{bmatrix} \sin(\theta_2) & \cos(\theta_2) & 0 & 0.235 \sin(\theta_2) \\ -\cos(\theta_2) & \sin(\theta_2) & 0 & -0.235 \cos(\theta_2) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_3^2 = \begin{bmatrix} \cos(\theta_3) & -\sin(\theta_3) & 0 & 0.230 \cos(\theta_3) \\ \sin(\theta_3) & \cos(\theta_3) & 0 & 0.230 \sin(\theta_3) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_4^3 = \begin{bmatrix} T_4^3 = \sin(\theta_4) & 0 & \cos(\theta_4) & 0 \\ -\cos(\theta_4) & 0 & \sin(\theta_4) & 0 \\ 0 & -1 & 0 & 0.085 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_5^4 = \begin{bmatrix} \cos(\theta_5) & 0 & \sin(\theta_5) & 0 \\ \sin(\theta_5) & 0 & -\cos(\theta_5) & 0 \\ 0 & 1 & 0 & 0.076 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T_{EndEffector}^5 = \begin{bmatrix} \cos(\theta_6) & 0 & -\sin(\theta_6) & 0 \\ \sin(\theta_6) & 0 & \cos(\theta_6) & 0 \\ 0 & -1 & 0 & 0.15878 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The total transformation between the base frame and the end effector is given by :

$$T_{EndEffector}^{Base} = T_1^{Base} T_2^1 T_3^2 T_4^3 T_5^4 T_{EndEffector}^5 \quad (1)$$

$\begin{aligned} &(((\sin(\theta_1)\sin(\theta_2)\cos(\theta_3) + \cos(\theta_1)\cos(\theta_2)\cos(\theta_3))\cos(\theta_4) + \\ &(-\sin(\theta_2)\cos(\theta_3)\cos(\theta_4) - \\ &\sin(\theta_1)\cos(\theta_2)\cos(\theta_3))\sin(\theta_4))\cos(\theta_5) + \\ &\sin(\theta_1)\sin(\theta_2)\cos(\theta_3) + (-\sin(\theta_1)\sin(\theta_2)\cos(\theta_3) + \\ &\cos(\theta_1)\cos(\theta_2)\cos(\theta_3))\sin(\theta_4) + (-\sin(\theta_2)\cos(\theta_3)\cos(\theta_4) - \\ &\sin(\theta_1)\cos(\theta_2)\cos(\theta_3))\sin(\theta_4))\sin(\theta_5) \end{aligned}$	$\begin{aligned} &(-\sin(\theta_1)\sin(\theta_2)\cos(\theta_3) + \\ &\cos(\theta_1)\cos(\theta_2)\cos(\theta_3))\cos(\theta_4) + (- \\ &\sin(\theta_2)\cos(\theta_3)\cos(\theta_4) - \\ &\sin(\theta_1)\cos(\theta_2)\cos(\theta_3))\sin(\theta_4))\sin(\theta_5) + \\ &\sin(\theta_1)\cos(\theta_2) \end{aligned}$	$\begin{aligned} &(-((\sin(\theta_1)\sin(\theta_2)\cos(\theta_3) + \cos(\theta_1)\cos(\theta_2)\cos(\theta_3))\cos(\theta_4) + \\ &(-\sin(\theta_2)\cos(\theta_3)\cos(\theta_4) - \\ &\sin(\theta_1)\cos(\theta_2)\cos(\theta_3))\sin(\theta_4))\cos(\theta_5) + \\ &\sin(\theta_1)\sin(\theta_2)\cos(\theta_3) + (-\sin(\theta_1)\sin(\theta_2)\cos(\theta_3) + \\ &\cos(\theta_1)\cos(\theta_2)\cos(\theta_3))\sin(\theta_4) + (-\sin(\theta_2)\cos(\theta_3)\cos(\theta_4) - \\ &\sin(\theta_1)\cos(\theta_2)\cos(\theta_3))\sin(\theta_4))\cos(\theta_5) \end{aligned}$	$\begin{aligned} &0.15878((-\sin(\theta_1)\sin(\theta_2)\cos(\theta_3) + \cos(\theta_1)\cos(\theta_2)\cos(\theta_3))\cos(\theta_4) + (- \\ &\sin(\theta_2)\cos(\theta_3)\cos(\theta_4) - \sin(\theta_1)\cos(\theta_2)\cos(\theta_3))\sin(\theta_4))\sin(\theta_5) - \\ &0.076(-\sin(\theta_2)\sin(\theta_3)\cos(\theta_4) + \cos(\theta_1)\cos(\theta_2)\cos(\theta_3))\sin(\theta_4) + \\ &0.076(-\sin(\theta_2)\cos(\theta_3)\cos(\theta_4) - \sin(\theta_1)\cos(\theta_2)\cos(\theta_3))\cos(\theta_5) - \\ &0.15878\sin(\theta_1)\cos(\theta_2) - 0.085\sin(\theta_1) - 0.23\sin(\theta_2)\sin(\theta_3)\cos(\theta_4) + \\ &0.23\cos(\theta_1)\cos(\theta_2)\cos(\theta_3) + 0.235\cos(\theta_1)\cos(\theta_2) \end{aligned}$
$\begin{aligned} &((- \sin(\theta_1)\sin(\theta_2)\sin(\theta_3) + \sin(\theta_1)\cos(\theta_2)\cos(\theta_3))\cos(\theta_4) + (- \\ &\sin(\theta_1)\sin(\theta_2)\cos(\theta_3) - \sin(\theta_1)\sin(\theta_2)\cos(\theta_3))\sin(\theta_4))\cos(\theta_5) - \\ &\sin(\theta_2)\cos(\theta_3)\cos(\theta_4) + (-\sin(\theta_1)\sin(\theta_2)\sin(\theta_3) + \\ &\sin(\theta_1)\cos(\theta_2)\cos(\theta_3))\sin(\theta_4) + (-\sin(\theta_1)\sin(\theta_2)\cos(\theta_3) - \\ &\sin(\theta_1)\sin(\theta_2)\cos(\theta_3))\cos(\theta_5) \end{aligned}$	$\begin{aligned} &(-(\sin(\theta_1)\sin(\theta_2)\sin(\theta_3) + \\ &\sin(\theta_1)\cos(\theta_2)\cos(\theta_3))\cos(\theta_4) + (- \\ &\sin(\theta_1)\sin(\theta_2)\cos(\theta_3) - \\ &\sin(\theta_1)\sin(\theta_2)\cos(\theta_3))\sin(\theta_4))\sin(\theta_5) - \\ &\cos(\theta_1)\cos(\theta_2) \end{aligned}$	$\begin{aligned} &(-((\sin(\theta_1)\sin(\theta_2)\sin(\theta_3) + \sin(\theta_1)\cos(\theta_2)\cos(\theta_3))\cos(\theta_4) + (- \\ &\sin(\theta_1)\sin(\theta_2)\cos(\theta_3) - \sin(\theta_1)\sin(\theta_2)\cos(\theta_3))\sin(\theta_4))\cos(\theta_5) - \\ &\sin(\theta_2)\cos(\theta_3)\cos(\theta_4) + (-\sin(\theta_1)\sin(\theta_2)\sin(\theta_3) + \\ &\sin(\theta_1)\cos(\theta_2)\cos(\theta_3))\sin(\theta_4) + (-\sin(\theta_1)\sin(\theta_2)\cos(\theta_3) - \\ &\sin(\theta_1)\sin(\theta_2)\cos(\theta_3))\cos(\theta_5) \end{aligned}$	$\begin{aligned} &0.15878((-\sin(\theta_1)\sin(\theta_2)\sin(\theta_3) + \sin(\theta_1)\cos(\theta_2)\cos(\theta_3))\cos(\theta_4) + (- \\ &\sin(\theta_1)\sin(\theta_2)\cos(\theta_3) - \sin(\theta_1)\sin(\theta_2)\cos(\theta_3))\sin(\theta_4))\sin(\theta_5) - 0.076(- \\ &\sin(\theta_1)\sin(\theta_2)\sin(\theta_3) + \sin(\theta_1)\cos(\theta_2)\cos(\theta_3))\sin(\theta_4) + 0.076(- \\ &\sin(\theta_1)\sin(\theta_2)\cos(\theta_3) - \sin(\theta_1)\sin(\theta_2)\cos(\theta_3))\cos(\theta_5) - \\ &0.23\sin(\theta_1)\sin(\theta_2)\sin(\theta_3) + 0.23\sin(\theta_1)\cos(\theta_2)\cos(\theta_3) + \\ &0.235\sin(\theta_1)\cos(\theta_2) + 0.15878\cos(\theta_1)\cos(\theta_2) + 0.085\cos(\theta_1) \end{aligned}$
$\begin{aligned} &((\sin(\theta_1)\sin(\theta_2) - \cos(\theta_1)\cos(\theta_2))\sin(\theta_4) + (-\sin(\theta_2)\cos(\theta_3) - \\ &\sin(\theta_1)\cos(\theta_2)\cos(\theta_3))\cos(\theta_4) + ((\sin(\theta_2)\sin(\theta_3) - \\ &\cos(\theta_1)\cos(\theta_2)\cos(\theta_3) - (-\sin(\theta_2)\cos(\theta_3) - \\ &\sin(\theta_1)\cos(\theta_2)\cos(\theta_3))\sin(\theta_4))\sin(\theta_5) \end{aligned}$	$\begin{aligned} &((\sin(\theta_1)\sin(\theta_2) - \cos(\theta_1)\cos(\theta_2))\sin(\theta_4) + (- \\ &\sin(\theta_2)\cos(\theta_3) - \sin(\theta_1)\cos(\theta_2)\cos(\theta_3))\cos(\theta_4) + (- \\ &\sin(\theta_2)\cos(\theta_3) - \sin(\theta_1)\cos(\theta_2)\cos(\theta_3))\sin(\theta_4))\sin(\theta_5) \end{aligned}$	$\begin{aligned} &(-(\sin(\theta_1)\sin(\theta_2) - \cos(\theta_1)\cos(\theta_2))\sin(\theta_4) + (-\sin(\theta_2)\cos(\theta_3) - \\ &\sin(\theta_1)\cos(\theta_2)\cos(\theta_3))\cos(\theta_4) + ((\sin(\theta_2)\sin(\theta_3) - \\ &\cos(\theta_1)\cos(\theta_2)\cos(\theta_3) - (-\sin(\theta_2)\cos(\theta_3) - \\ &\sin(\theta_1)\cos(\theta_2)\cos(\theta_3))\sin(\theta_4))\cos(\theta_5) \end{aligned}$	$\begin{aligned} &0.15878((\sin(\theta_1)\sin(\theta_2) - \cos(\theta_1)\cos(\theta_2))\sin(\theta_4) + (-\sin(\theta_2)\cos(\theta_3) - \\ &\sin(\theta_1)\cos(\theta_2)\cos(\theta_3))\cos(\theta_4) + 0.076(\sin(\theta_2)\sin(\theta_3) - \\ &\cos(\theta_1)\cos(\theta_2)\cos(\theta_3) - 0.076(-\sin(\theta_2)\cos(\theta_3) - \\ &\sin(\theta_1)\cos(\theta_2)\cos(\theta_3))\sin(\theta_4) - 0.23\sin(\theta_1)\cos(\theta_2) - 0.235\sin(\theta_1) - \\ &0.23\sin(\theta_2)\cos(\theta_3) + 0.076 \end{aligned}$
0	0	0	1

Figure 6: Final Transformation

8 Forward Kinematics Validation

The validation of derived $T_{EndEffector}^{Base}$ from DH table is done in MATLAB using Peter Corke's Robotics toolkit. Numerical results together with visual plot of position and orientation of a robot in MATLAB environment gives clear insight of the kinematic behaviour of a robot.

The below section shows our validation of EB100's Forward kinematics via DH table. In each test case joint angles were known and applied to the manipulator. Position of End-Effector from Peter Corke's was then evaluated with the transformation matrix $T_{EndEffector}^{Base}$ derived from DH parameters.

Test cases are as follows

- End Effector position at home position i.e. all angles are 0 rad.
- End Effector position when θ_1 is $-\pi/2$ rad.
- End Effector position when θ_2 is $-\pi/2$ rad.
- End Effector position when θ_3 is $-\pi/2$ rad.
- End Effector position when θ_4 is $-\pi/2$ rad.
- End Effector position when θ_5 is $-\pi/2$ rad.

Transformation matrices for above test cases are

$$T_{EndEffector}^0 = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0.24378 \\ 0 & 0 & 1 & 0.617 \\ 0 & 0 & 0 & 1.0 \end{bmatrix}$$

home position i.e. all angles are 0 rad.

$$T_{EndEffector}^0 = \begin{bmatrix} 0 & -1 & 0 & 0.24378 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0.617 \\ 0 & 0 & 0 & 1.0 \end{bmatrix}$$

θ_1 is $-\pi/2$ rad.

$$T_{EndEffector}^0 = \begin{bmatrix} 0 & 0 & -1 & -0.541 \\ 0 & -1 & 0 & 0.24378 \\ -1 & 0 & 0 & 0.076 \\ 0 & 0 & 0 & 1.0 \end{bmatrix}$$

θ_2 is $-\pi/2$ rad.

$$T_{EndEffector}^0 = \begin{bmatrix} 0 & 0 & -1 & -0.306 \\ 0 & -1 & 0 & 0.24378 \\ -1 & 0 & 0 & 0.311 \\ 0 & 0 & 0 & 1.0 \end{bmatrix}$$

θ_3 is $-\pi/2$ rad.

$$T_{EndEffector}^0 = \begin{bmatrix} 0 & 0 & -1 & -0.076 \\ 0 & -1 & 0 & 0.24378 \\ -1 & 0 & 0 & 0.541 \\ 0 & 0 & 0 & 1.0 \end{bmatrix}$$

θ_4 is $-\pi/2$ rad.

$$T_{EndEffector}^0 = \begin{bmatrix} 0 & -1 & 0 & 0.15878 \\ 1 & 0 & 0 & 0.085 \\ 0 & 0 & 1 & 0.617 \\ 0 & 0 & 0 & 1.0 \end{bmatrix}$$

θ_5 is $-\pi/2$ rad.

8.1 Verification with Peter Corke's Robotics Toolbox for MATLAB

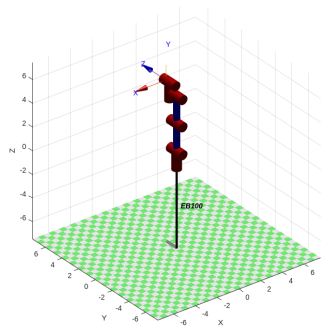
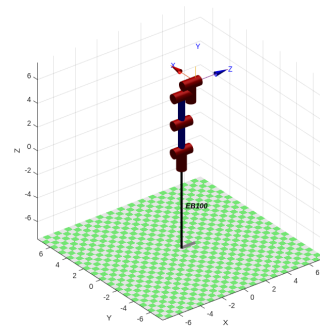
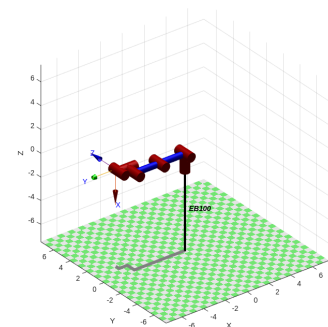
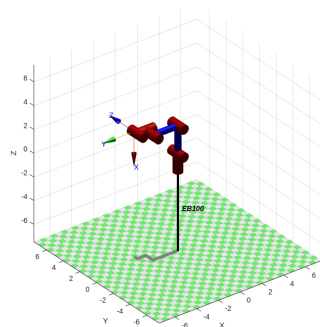
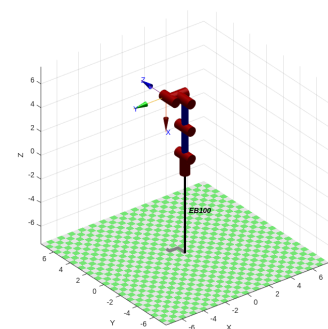
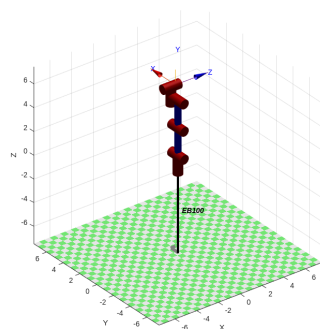


Figure 7: Home Position

Figure 8: When $\theta_1 = -\pi/2$ Figure 9: When $\theta_2 = -\pi/2$ Figure 10: When $\theta_3 = -\pi/2$ Figure 11: When $\theta_4 = -\pi/2$ Figure 12: When $\theta_5 = -\pi/2$

The above results validate our forward kinematics model for EB100 as transformation matrices of test cases match with peter corke's toolkit visualization and position values.

9 Inverse Kinematics

As opposed to forward kinematics, which computes the workspace coordinates of the robot given a configuration as input, inverse kinematics (IK) is essentially the reverse operation: computing configuration(s) to reach a desired workspace coordinate. However, unlike forward kinematics, inverse kinematics cannot be solved in a closed-form expression (in general). We shall see there may be no solutions, multiple solutions, or even an infinite number of solutions to an IK problem. The solutions which ensure collision-free free operation and minimum joint motion are considered more optimum.

Analytical approach has been followed to develop IK model of EB100. This approach ensures that for any object within EB100 workspace, the model determines correct joint angles. The joint angles θ_1 , θ_2 , θ_4 , θ_5 and θ_6 are calculated using this approach while End Effector position is derived from perception module and orientation is always parallel to table.

The relationship between end-effector velocities $\dot{x}$ and joint velocities $\dot{q}$ is given by

$$\dot{x} = J^{-1}(q)\dot{q}$$

where J^{-1} is Jacobian inverse.

Singularity occurs when the task has an infinite number of solutions, or precisely one solution. In redundant IK, a singularity is observed as a change in the intrinsic dimensionality of the solution set. Singularities are unstable in the sense that any open neighborhood containing a singular task point (x) will have different numbers of solutions. In other words, if x were perturbed slightly, the number of solutions will almost certainly change. Non-singular points do not have this problem: for a non-singular point x , there will exist an open neighborhood of x for which the number of solutions remains constant. In practical terms, singularities usually avoided during IK for a number of reasons: as a singularity is approached in task space, the solution configurations change increasingly rapidly, and the solution set may change size, making it challenging to produce smooth movements. How singularities are handled in IK solver in this project is discussed more in section 17.1.1.

10 Inverse Kinematics Validation

Inverse kinematics was validated using a method similar to Homework-2/3. For a given shape, a trajectory was calculated as a function of time. End effector velocities were calculated from the trajectory, and joint angles were calculated from the velocities using simple integration. These angles were then used to calculate the end effector position using forward kinematics transformation as discussed above. As can be seen from the results the End effector follows the desired trajectory which validates our IK model for EB100.

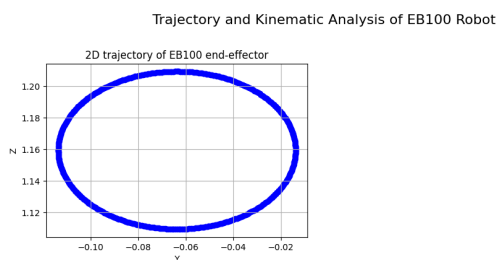


Figure 13: Desired trajectory of End Effector

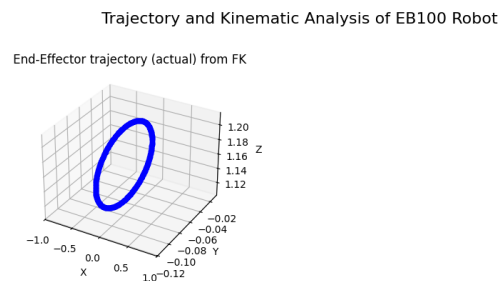


Figure 14: Actual trajectory of End Effector

11 Velocity Kinematics – EB100

11.1 Calculating the 0Z_n of the Transformations

The transformations are calculated by the Python code given, and these components were extracted and are presented here.

$$T_1^0 = T_1^0$$

$${}^0Z_1 = \begin{bmatrix} -\sin(\theta_1) \\ \cos(\theta_1) \\ 0 \end{bmatrix}$$

$$T_2^0 = T_1^0 T_2^1$$

$${}^0Z_2 = \begin{bmatrix} -\sin(\theta_1) \\ \cos(\theta_1) \\ 0 \end{bmatrix}$$

$$T_3^0 = T_1^0 T_2^1 T_3^2$$

$${}^0Z_3 = \begin{bmatrix} -\sin(\theta_1) \\ \cos(\theta_1) \\ 0 \end{bmatrix}$$

$$T_4^0 = T_1^0 T_2^1 T_3^2 T_4^3$$

$${}^0Z_4 = \begin{bmatrix} -\sin(\theta_2 + \theta_3 + \theta_4) \cos(\theta_1) \\ -\sin(\theta_1) \sin(\theta_2 + \theta_3 + \theta_4) \\ \cos(\theta_2 + \theta_3 + \theta_4) \end{bmatrix}$$

$$T_5^0 = T_1^0 T_2^1 T_3^2 T_4^3 T_5^4$$

$${}^0Z_5 = \begin{bmatrix} \sin(\theta_1) \cos(\theta_5) + \sin(\theta_5) \cos(\theta_1) \cos(\theta_2 + \theta_3 + \theta_4) \\ \sin(\theta_1) \sin(\theta_5) \cos(\theta_2 + \theta_3 + \theta_4) - \cos(\theta_1) \cos(\theta_5) \\ \sin(\theta_5) \sin(\theta_2 + \theta_3 + \theta_4) \end{bmatrix}$$

$$T_6^0 = T_1^0 T_2^1 T_3^2 T_4^3 T_5^4 T_6^5$$

$${}^0Z_6 = \begin{bmatrix} \sin(\theta_1) \cos(\theta_6) + \sin(\theta_6) \cos(\theta_1) \cos(\theta_2 + \theta_3 + \theta_4) \\ \sin(\theta_1) \sin(\theta_6) \cos(\theta_2 + \theta_3 + \theta_4) - \cos(\theta_1) \cos(\theta_6) \\ \sin(\theta_6) \sin(\theta_2 + \theta_3 + \theta_4) \end{bmatrix}$$

The end effector's position can be found in the last part of this combined matrix by looking at the last column to find the x, y, z coordinates, which show where the end effector is located in space. This extraction was done by Python, yielding the result $[x, y, z]$.

$$\begin{bmatrix} -0.15878 \sin(\theta_1) \cos(\theta_5) - 0.085 \sin(\theta_1) + 0.15878 \sin(\theta_5) \cos(\theta_1) \cos(\theta_2 + \theta_3 + \theta_4) - 0.076 \sin(\theta_2 + \theta_3 + \theta_4) \cos(\theta_1) + 0.235 \cos(\theta_1) \cos(\theta_2) + 0.23 \cos(\theta_1) \cos(\theta_2 + \theta_3) \\ 0.15878 \sin(\theta_1) \sin(\theta_5) \cos(\theta_2 + \theta_3 + \theta_4) - 0.076 \sin(\theta_1) \sin(\theta_2 + \theta_3 + \theta_4) + 0.235 \sin(\theta_1) \cos(\theta_2) + 0.23 \sin(\theta_1) \cos(\theta_2 + \theta_3) + 0.15878 \cos(\theta_1) \cos(\theta_5) + 0.085 \cos(\theta_1) \\ -0.235 \sin(\theta_2) - 0.15878 \sin(\theta_5) \sin(\theta_2 + \theta_3 + \theta_4) - 0.23 \sin(\theta_2 + \theta_3) - 0.076 \cos(\theta_2 + \theta_3 + \theta_4) + 0.076 \end{bmatrix}$$

Figure 15: End effector position

The other part of the Jacobian is found by considering the partial derivatives of x, y , and z with respect to θ_1 up to θ_n (where n is six in this case).

$$x = f_1(\theta_1, \theta_2, \dots, \theta_n)$$

$$y = f_2(\theta_1, \theta_2, \dots, \theta_n)$$

$$z = f_3(\theta_1, \theta_2, \dots, \theta_n)$$

The Jacobian matrix, J , relates changes in joint parameters to changes in the position of the end effector. It is computed as follows:

Form the Jacobian matrix by taking the partial derivatives of the end effector position with respect to each joint variable. The Jacobian will have three rows (for x, y, z) and n columns (for $\theta_1, \theta_2, \dots, \theta_n$):

$$J_p = \begin{bmatrix} \frac{\partial x}{\partial \theta_1} & \frac{\partial x}{\partial \theta_2} & \frac{\partial x}{\partial \theta_3} & \frac{\partial x}{\partial \theta_4} & \frac{\partial x}{\partial \theta_5} & \frac{\partial x}{\partial \theta_6} \\ \frac{\partial y}{\partial \theta_1} & \frac{\partial y}{\partial \theta_2} & \frac{\partial y}{\partial \theta_3} & \frac{\partial y}{\partial \theta_4} & \frac{\partial y}{\partial \theta_5} & \frac{\partial y}{\partial \theta_6} \\ \frac{\partial z}{\partial \theta_1} & \frac{\partial z}{\partial \theta_2} & \frac{\partial z}{\partial \theta_3} & \frac{\partial z}{\partial \theta_4} & \frac{\partial z}{\partial \theta_5} & \frac{\partial z}{\partial \theta_6} \end{bmatrix}$$

J_p is then combined with the 0Z_n to result in a Jacobian matrix like the following:

$${}^0\mathbf{J} = \begin{pmatrix} \frac{\partial x}{\partial \theta_1} & \frac{\partial x}{\partial \theta_2} & \frac{\partial x}{\partial \theta_3} & \frac{\partial x}{\partial \theta_4} & \frac{\partial x}{\partial \theta_5} & \frac{\partial x}{\partial \theta_6} \\ \frac{\partial y}{\partial \theta_1} & \frac{\partial y}{\partial \theta_2} & \frac{\partial y}{\partial \theta_3} & \frac{\partial y}{\partial \theta_4} & \frac{\partial y}{\partial \theta_5} & \frac{\partial y}{\partial \theta_6} \\ \frac{\partial z}{\partial \theta_1} & \frac{\partial z}{\partial \theta_2} & \frac{\partial z}{\partial \theta_3} & \frac{\partial z}{\partial \theta_4} & \frac{\partial z}{\partial \theta_5} & \frac{\partial z}{\partial \theta_6} \\ {}^0\mathbf{Z}_1 & {}^0\mathbf{Z}_2 & {}^0\mathbf{Z}_3 & {}^0\mathbf{Z}_4 & {}^0\mathbf{Z}_5 & {}^0\mathbf{Z}_6 \end{pmatrix}$$

Jacobian in parametric form

$$\begin{aligned}
& -0.235 \sin(\theta_1) \sin(\theta_2) + 0.15878 \sin(\theta_1) \sin(\theta_5) \cos(\theta_2 + \theta_3 + \theta_4) - 0.23 \sin(\theta_1) \sin(\theta_2 + \theta_3) - 0.076 \sin(\theta_1) \sin(\theta_2 + \theta_3 + \theta_4) - 0.15878 \cos(\theta_1) \cos(\theta_5) - 0.085 \cos(\theta_1) \\
& -0.15878 \sin(\theta_1) \cos(\theta_5) - 0.085 \sin(\theta_1) + 0.235 \sin(\theta_2) \cos(\theta_1) - 0.15878 \sin(\theta_5) \cos(\theta_1) \cos(\theta_2 + \theta_3 + \theta_4) + 0.23 \sin(\theta_2 + \theta_3) \cos(\theta_1) + 0.076 \sin(\theta_2 + \theta_3 + \theta_4) \cos(\theta_1) \\
& \quad 0 \\
& \quad -\sin(\theta_1) \\
& \quad \cos(\theta_1) \\
& \quad 0
\end{aligned}$$

(a) Jacobian column 1

$$\begin{aligned}
& (0.15878 \sin(\theta_5) \sin(\theta_2 + \theta_3 + \theta_4) + 0.235 \cos(\theta_2) + 0.23 \cos(\theta_2 + \theta_3) + 0.076 \cos(\theta_2 + \theta_3 + \theta_4)) \cos(\theta_1) \\
& (0.15878 \sin(\theta_5) \sin(\theta_2 + \theta_3 + \theta_4) + 0.235 \cos(\theta_2) + 0.23 \cos(\theta_2 + \theta_3) + 0.076 \cos(\theta_2 + \theta_3 + \theta_4)) \sin(\theta_1) \\
& \quad -0.235 \sin(\theta_2) + 0.15878 \sin(\theta_5) \cos(\theta_2 + \theta_3 + \theta_4) - 0.23 \sin(\theta_2 + \theta_3) - 0.076 \sin(\theta_2 + \theta_3 + \theta_4) \\
& \quad -\sin(\theta_1) \\
& \quad \cos(\theta_1) \\
& \quad 0
\end{aligned}$$

(b) Jacobian column 2

$$\begin{aligned}
& (0.15878 \sin(\theta_5) \sin(\theta_2 + \theta_3 + \theta_4) + 0.23 \cos(\theta_2 + \theta_3) + 0.076 \cos(\theta_2 + \theta_3 + \theta_4)) \cos(\theta_1) \\
& (0.15878 \sin(\theta_5) \sin(\theta_2 + \theta_3 + \theta_4) + 0.23 \cos(\theta_2 + \theta_3) + 0.076 \cos(\theta_2 + \theta_3 + \theta_4)) \sin(\theta_1) \\
& \quad 0.15878 \sin(\theta_5) \cos(\theta_2 + \theta_3 + \theta_4) - 0.23 \sin(\theta_2 + \theta_3) - 0.076 \sin(\theta_2 + \theta_3 + \theta_4) \\
& \quad -\sin(\theta_1) \\
& \quad \cos(\theta_1) \\
& \quad 0
\end{aligned}$$

(c) Jacobian column 3

$$\begin{aligned}
& (0.15878 \sin(\theta_5) \sin(\theta_2 + \theta_3 + \theta_4) + 0.076 \cos(\theta_2 + \theta_3 + \theta_4)) \cos(\theta_1) \\
& (0.15878 \sin(\theta_5) \sin(\theta_2 + \theta_3 + \theta_4) + 0.076 \cos(\theta_2 + \theta_3 + \theta_4)) \sin(\theta_1) \\
& \quad 0.15878 \sin(\theta_5) \cos(\theta_2 + \theta_3 + \theta_4) - 0.076 \sin(\theta_2 + \theta_3 + \theta_4) \\
& \quad \sin(\theta_2 + \theta_3 + \theta_4) \cos(\theta_1) \\
& \quad \sin(\theta_1) \sin(\theta_2 + \theta_3 + \theta_4) \\
& \quad \cos(\theta_2 + \theta_3 + \theta_4)
\end{aligned}$$

(d) Jacobian column 4

$$\begin{aligned}
& 0.15878 \sin(\theta_1) \sin(\theta_5) - 0.15878 \cos(\theta_1) \cos(\theta_5) \cos(\theta_2 + \theta_3 + \theta_4) \\
& -0.15878 \sin(\theta_1) \cos(\theta_5) \cos(\theta_2 + \theta_3 + \theta_4) - 0.15878 \sin(\theta_5) \cos(\theta_1) \\
& \quad 0.15878 \sin(\theta_2 + \theta_3 + \theta_4) \cos(\theta_5) \\
& \quad -\sin(\theta_1) \cos(\theta_5) - \sin(\theta_5) \cos(\theta_1) \cos(\theta_2 + \theta_3 + \theta_4) \\
& \quad -\sin(\theta_1) \sin(\theta_5) \cos(\theta_2 + \theta_3 + \theta_4) + \cos(\theta_1) \cos(\theta_5) \\
& \quad \sin(\theta_5) \sin(\theta_2 + \theta_3 + \theta_4)
\end{aligned}$$

(e) Jacobian column 5

$$\begin{aligned}
& 0 \\
& 0 \\
& 0 \\
& (-\sin(\theta_1) \sin(\theta_5) + \cos(\theta_1) \cos(\theta_5) \cos(\theta_2 + \theta_3 + \theta_4)) \sin(\theta_6) + \sin(\theta_2 + \theta_3 + \theta_4) \cos(\theta_1) \cos(\theta_6) \\
& (\sin(\theta_1) \cos(\theta_5) \cos(\theta_2 + \theta_3 + \theta_4) + \sin(\theta_5) \cos(\theta_1)) \sin(\theta_6) + \sin(\theta_1) \sin(\theta_2 + \theta_3 + \theta_4) \cos(\theta_6) \\
& \quad -\sin(\theta_6) \sin(\theta_2 + \theta_3 + \theta_4) \cos(\theta_5) + \cos(\theta_6) \cos(\theta_2 + \theta_3 + \theta_4)
\end{aligned}$$

(f) Jacobian column 6

12 Workspace Study

Workspace study and reachability analysis are critical aspects of designing and testing robotic manipulators. They help determine the volume within which the robot can operate and validate whether specific target points lie within its reachable workspace. This section explains the methodology used for the EB100 manipulator to perform a workspace study using Python coding.

12.1 Denavit-Hartenberg (DH) Parameters

The manipulator's kinematics are defined using the Denavit-Hartenberg (DH) parameters, which describe the geometric relationships between the manipulator's joints. For the EB100 manipulator, the DH parameters include:

- a : Link length (distance along the x-axis between two consecutive joints).
- α : Link twist (angle between z-axes of consecutive joints).
- d : Link offset (distance along the z-axis between two consecutive joints).
- θ : Joint angle (rotational parameter, variable for revolute joints).

In the code, the DH parameters are predefined for all six joints of the EB100 manipulator. These parameters are used to calculate the robot's forward kinematics and workspace reach.

12.2 Workspace Reachability Calculation

The `workspaceSanityCheck` function evaluates whether a specified point in 3D space (x, y, z) lies within the robot's reachable workspace. The steps for this calculation are as follows:

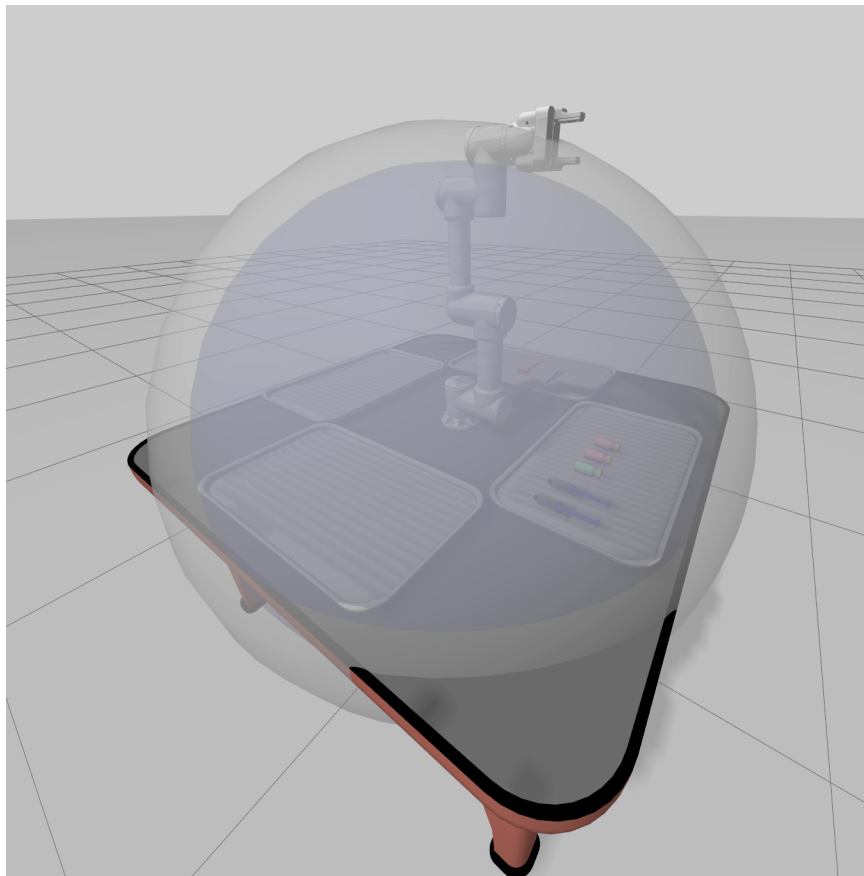


Figure 17: Workspace Study

White Region = Maximum reachable space, Blue Region = Reachable in any orientation

1. **Calculate Maximum Reach:** The maximum reach of the manipulator is estimated as the sum of the absolute values of the link lengths (a) and link offsets (d). This provides a scalar approximation of the robot's operational envelope:

$$\text{Max Reach} = \sum_{i=1}^n \left(\frac{|a_i|}{10} + \frac{|d_i|}{10} \right)$$

where n is the number of joints, and the values are scaled appropriately.

2. **Compute Distance to the Point:** The Euclidean distance from the manipulator's base to the target point (x, y, z) is calculated:

$$\text{Distance to Point} = \sqrt{x^2 + y^2 + z^2}$$

3. **Compare Distances:** The distance to the point is compared with the maximum reach. If the distance is less than or equal to the maximum reach, the point is considered reachable; otherwise, it is outside the workspace.
4. **Output Results:** The function logs information about the target point, the computed distances, and whether the point is reachable.

12.3 Challenges and Observations

- **Simplified Approximation:** The maximum reach calculation assumes a linear addition of link lengths and offsets, ignoring joint constraints and rotational limits. While this provides a quick estimate, it does not account for the actual kinematic configuration.
- **Geometric Constraints:** The presence of singularities or joint limits can reduce the effective workspace, which may result in points within the calculated maximum reach being unreachable in practice.
- **Validation and Testing:** The command-line implementation allows users to input arbitrary points and validate reachability, enabling iterative testing and refinement of the manipulator's configuration.

12.4 Conclusion

The workspace study and reachability calculation provide an initial estimate of the robot's operational limits, guiding both design and application. The use of DH parameters for geometric modeling and reachability analysis ensures that the manipulator can effectively interact with its environment while identifying limitations for further optimization.

13 Assumptions

The following assumptions were considered for the modeling and design of the robotic system for surgical tool sorting. These assumptions allowed us to simplify the design and focus on the primary objectives of the system:

1. **Sterile Environment:** The robot is assumed to operate in a sterile and controlled environment to ensure the safety and hygiene necessary for handling surgical instruments. All objects the robot interacts with, including tools and storage bins, are either sterilizable or disposable. This assumption eliminates the complexities introduced by external contaminants, environmental changes, or noise, ensuring the robot's operations remain consistent and predictable. Such conditions are essential for healthcare applications, where even minor contamination could compromise patient safety. It also simplifies the design by reducing the need for cleaning mechanisms on the robot.
2. **Known Tool Properties:** The physical characteristics of surgical tools, including weight, dimensions, and center of mass, are considered known beforehand. These parameters are essential for designing gripping strategies and ensuring stable and reliable tool handling. Additionally, tools are assumed to be rigid, which eliminates the need for modeling deformations or flexible materials. This assumption allows the robot to handle a limited and predefined set of tools with high precision and minimizes uncertainties in performance.
3. **Controlled Environment:** The robot operates in a controlled environment where factors like wear, friction, and noise are considered negligible. This assumption replicates real-world settings where surgical tools are handled in clean, stable, and quiet operating rooms. It simplifies the modeling process by allowing the focus to remain on the robot's functionality rather than external influences, ensuring more straightforward simulations and hardware implementation.
4. **Surgical Tools Are Clearly Identifiable:** Each tool is designed to have a unique appearance, enabling the robot's sensing system to categorize them accurately. This assumption simplifies visual recognition tasks, reduces computational overhead, and prevents the complexities of misidentification. As a result, the robot can reliably identify and handle the tools required for specific surgical procedures.
5. **No Overlap Between Objects:** Tools are arranged in the workspace without overlapping, ensuring that the robot's sensors can detect each tool individually. Overlapping tools often cause occlusion and clustering issues, complicating the detection process. By avoiding such scenarios, this assumption allows the focus to remain on the robot's ability to recognize and manipulate individual tools effectively.
6. **Constant Velocity:** The robot's movements between points are assumed to occur at a constant velocity. This simplifies trajectory planning and eliminates the need to model acceleration and deceleration phases. Such an assumption is practical for repetitive pick-and-place operations where consistent speed enhances efficiency and simplifies control algorithms.
7. **Gripper Reliability:** The gripper is considered reliable enough to grasp tools securely without requiring additional feedback mechanisms like slip detection. The force required for gripping is calibrated in advance to ensure consistent handling across all tool types. This assumption reduces the complexity of integrating force sensors or slip detection algorithms.
8. **Simple Control System:** A basic proportional-integral-derivative (PID) controller is assumed to be sufficient for managing the robot's motion. By focusing on PID control, the system avoids the complexities associated with advanced control algorithms while maintaining stable and accurate performance.
9. **Ideal Actuators:** The actuators are assumed to be ideal, meaning they operate without friction, wear, or noise. This assumption streamlines the dynamic and kinematic analysis required for system design and allows for more accurate simulations without introducing unnecessary variables.

These assumptions provided a clear framework for developing the robotic system. By focusing on a simplified yet practical approach, we ensured that the design remained efficient, precise, and suited for the sterile environment of surgical tool sorting. As the project progresses into hardware implementation, these assumptions will guide further refinements and adaptations to real-world conditions.

14 Control Method

The control methods implemented for the EB100 manipulator in this project focused on achieving precise, stable, and responsive motion for a variety of tasks, including surgical tool sorting. The control architecture integrates kinematic and dynamic modeling with feedback mechanisms to ensure robust performance under varying conditions.

14.1 Feedback Control Loop

At the heart of the system lies a closed-loop feedback controller. The desired joint positions, velocities, and accelerations ($q_d, \dot{q}_d, \ddot{q}_d$) are compared with the actual values ($q_a, \dot{q}_a, \ddot{q}_a$) obtained from the manipulator's sensors. The error signal generated is fed into a Proportional-Integral-Derivative (PID) controller, which calculates the required torques (τ) to minimize these errors. This feedback mechanism ensures that the manipulator tracks the desired trajectory accurately and compensates for disturbances in real time.

14.2 Dynamic Modeling and Control

The robot manipulator's dynamics are modeled to account for the interplay of joint torques, inertial forces, and external loads. The computed torques are applied to the manipulator to achieve the desired motion while maintaining stability. The dynamic model incorporates the payload capacity and frictional forces at the joints, ensuring accurate control even during pick-and-place operations.

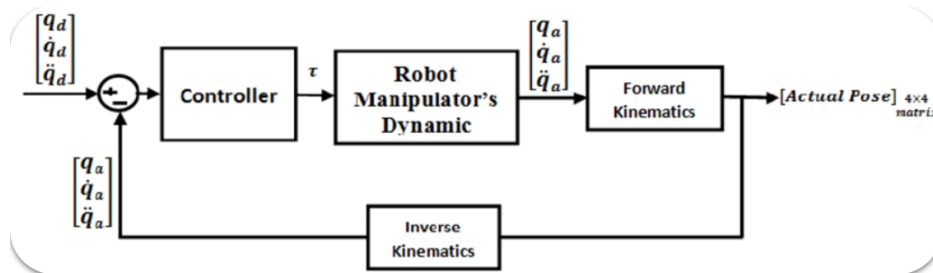


Figure 18: Robot Control

14.3 Integration with ROS

The control methods are integrated with the Robot Operating System (ROS) to facilitate seamless communication and interaction between the manipulator and the simulation environment. ROS topics are used to send and receive joint state information, while service calls handle trajectory execution and task coordination.

14.4 Challenges and Solutions

One of the key challenges was tuning the PID controllers to achieve smooth and stable motion. Iterative testing was conducted to fine-tune the gains for each joint. Additionally, real-time performance was optimized by reducing computational overhead through multiprocessing techniques for Jacobian calculations.

In conclusion, the implemented control methods provided the EB100 manipulator with the precision and stability required for surgical tool sorting tasks in a simulated environment. These methods also laid the foundation for transitioning to real-world hardware implementation.

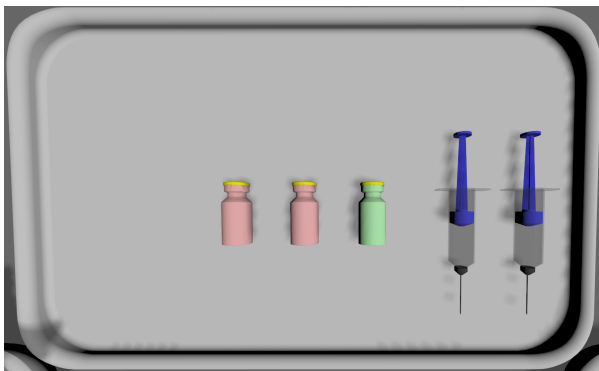
15 Perception Module

In our system, the Perception Module plays a pivotal role in recognizing the tools placed on the tray, enabling the robot to interact with them efficiently. This module utilizes OpenCV combined with the YOLOv8x object detection model to identify various tools on the tray and extract essential information such as object labels, centroids, and bounding boxes. The data captured through this process is then utilized to determine the position and orientation of each tool in the global frame via transformation matrices, facilitating precise manipulation.

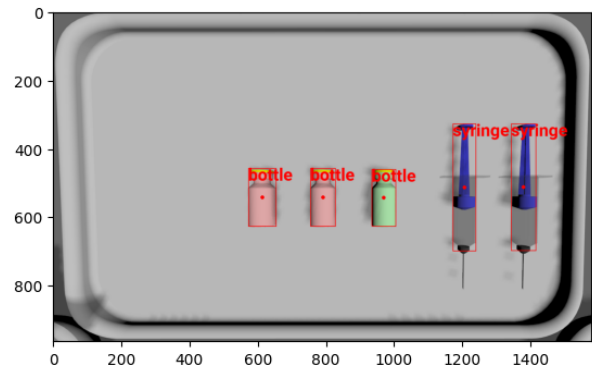
15.1 Tool Detection Process

Upon receiving an order, the robot autonomously moves to a preset location positioned 20 cm above the tool tray. At this location, the robot takes a high-resolution image of the tray, ensuring that the camera captures the full arrangement of tools. The captured image is then processed using the YOLOv8x object detection model, which outputs the following key information for each identified tool:

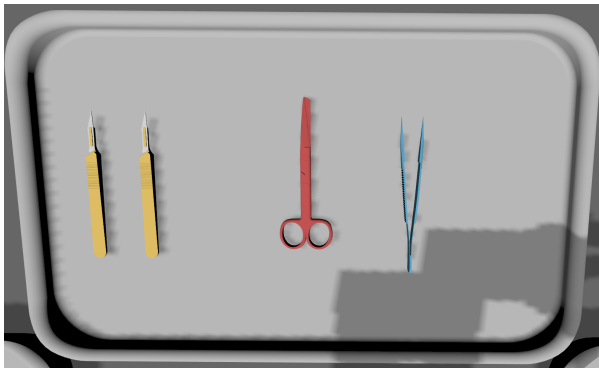
- **Label:** The specific type of tool (e.g., screwdriver, wrench, etc.).
- **Centroid:** The center point of the tool in pixel coordinates, used to determine its location relative to the camera.
- **Bounding Box:** A rectangle enclosing the detected tool, providing the dimensions and position of the object within the image.



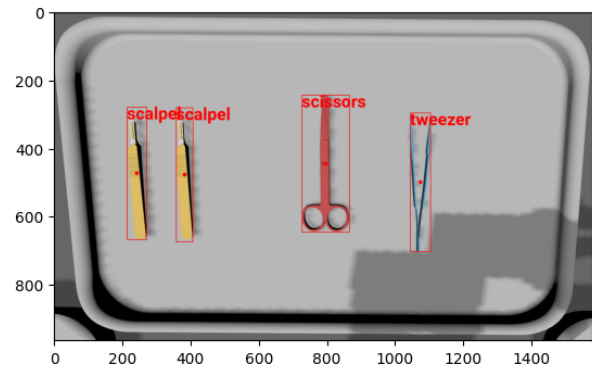
(a) Sample Disposable Tray



(b) Detected Tools with Bounding Boxes and Labels



(c) Sample Reusable Tray



(d) Detected Tools with Bounding Boxes and Labels

Figure 19: Example Source Trays

15.2 Data Integration and Pose Estimation

Once the object detection process is complete, the perception module compiles the data into a structured format and generates a memory map of the tools. This map is crucial for the subsequent decision-making process, where the robot can cross-reference the detected tools with the task requirements (e.g., picking up specific tools).

To determine the pose of each tool in the global frame, the centroid data, along with the bounding box information, is used to calculate the transformation matrices. These matrices are essential for transforming the 2D coordinates from the image plane to 3D coordinates in the robot's workspace, enabling accurate positioning and manipulation of each tool.

15.3 Cross-Referencing with Task Order

Once the tool data is processed and the poses are calculated, the system utilizes the memory map for selecting the appropriate tool(s) required for the order. By cross-referencing the object labels with the robot's task instructions, the perception module ensures that the correct tool is picked for each step of the operation. The identified tools' pose data is subsequently fed into the robot's motion planning module, guiding the robot to the precise location for pick-and-place actions.

16 Gazebo & RViz Visualization

16.1 Gazebo Plugins

16.1.1 EB100 Manipulator Simulation

The integration of the EB100 manipulator into the Gazebo environment was a key step in simulating its behavior and capabilities. The EB100 is a 6-DOF robotic arm, and its accurate representation in Gazebo required detailed modeling of its joints, links, and dynamics. This simulation supports the development and testing of control strategies, trajectory planning, and interaction with the environment.

To implement the EB100 manipulator, we modified the Unified Robot Description Format (URDF) file, incorporating a `<gazebo>` tag for each joint and link to ensure proper simulation in Gazebo.

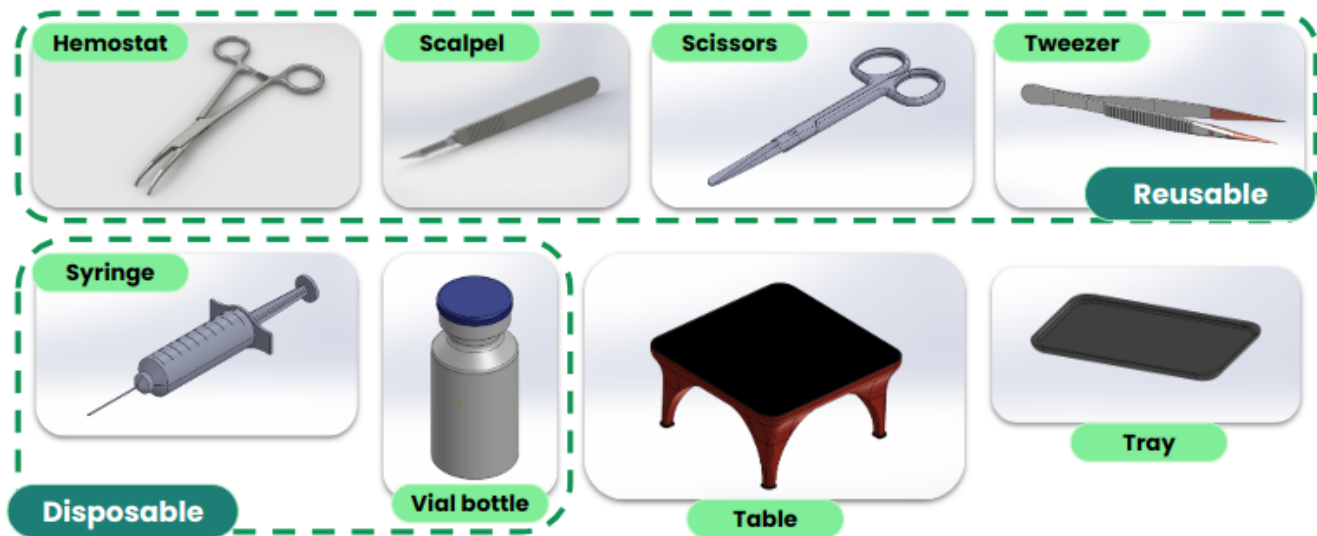


Figure 20: Surrounding Items visualization in RViz

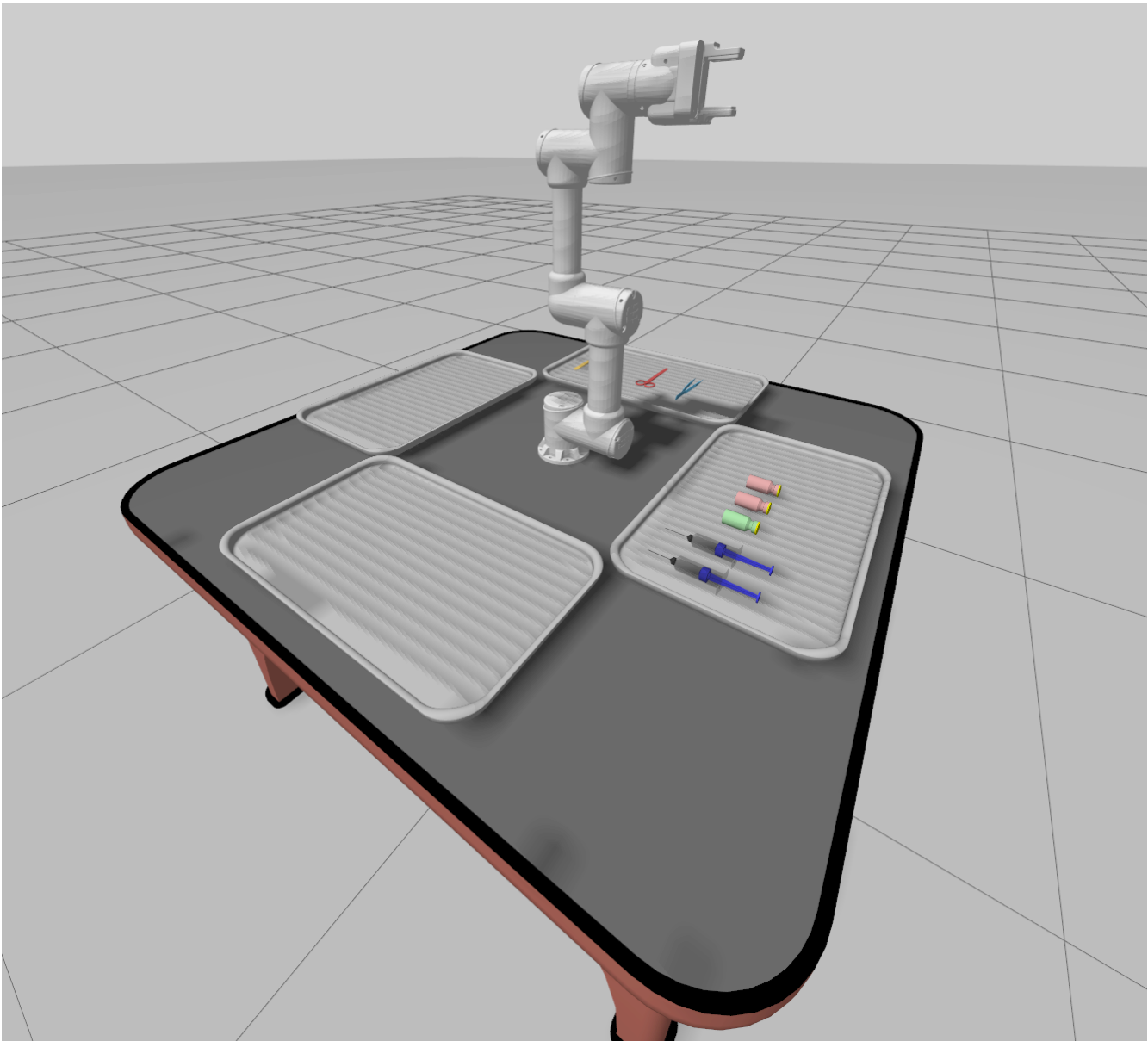


Figure 21: EB100 Manipulator visualization in Ignition Gazebo

16.1.2 Key Components

- **Joints:** Each joint was assigned a PID controller to ensure stable and responsive motion.
- **Dynamics:** The simulation accounted for the manipulator's payload capacity, joint limits, and dynamic properties like inertia and friction.
- **Control Interface:** ROS topics were defined for controlling joint positions, velocities, and torques.
- **Gripper Integration:** The EB100 manipulator's gripper was configured to interact with objects in the environment, enabling pick-and-place operations.

16.1.3 Challenges and Mitigations

- **Joint Control:** Initially, achieving stable joint motion was challenging due to inaccurate PID tuning. This was mitigated by iterative testing and fine-tuning of PID parameters for each joint.

- **Coordinate System Alignment:** Discrepancies in link alignments caused issues during the initial stages of simulation. Careful adjustments to the URDF model resolved these misalignments.
- **Real-Time Simulation Factor:** The computational load of simulating a 6-DOF manipulator impacted real-time performance. Optimizing the physics engine settings in Gazebo and reducing unnecessary collision elements improved simulation efficiency.

16.2 RViz Visualization

To visualize the EB100 manipulator in RViz, the following steps were taken:

- Add a **RobotModel** display and load the URDF file to represent the manipulator.
- Publish joint states using the *joint_state_publisher* node to animate the manipulator in real-time.
- Set the Fixed Frame to *base_link* to ensure proper alignment with the manipulator's coordinate system.

By accurately simulating the EB100 manipulator, the Gazebo environment facilitated the testing of control algorithms and task execution strategies, paving the way for successful hardware implementation.

17 Problems Faced

17.1 Singularities in 6DOF Robot Arm

Singularities are configurations in a robotic manipulator where the Jacobian matrix loses rank, leading to a loss of certain degrees of freedom. This condition causes numerical instabilities and can result in unexpected or unfeasible motion. For a 6DOF robotic arm, singularities are often categorized into the following types:

- **Wrist Singularities:** Occur when the wrist's axes align, making it impossible to determine the unique rotation about these axes.
- **Shoulder Singularities:** Occur when the arm stretches fully, aligning the shoulder and wrist axes.
- **Elbow Singularities:** Happen when the elbow is fully extended or folded, resulting in a collinear configuration.

17.1.1 Addressing Singularities

To handle singularities during inverse kinematics computations, we adopted the following approaches:

1. **Pseudo-Inverse of the Jacobian:** The pseudo-inverse method provides a robust solution by computing a generalized inverse of the Jacobian matrix. This technique minimizes joint velocities in the least-squares sense, ensuring numerical stability even in the vicinity of singular configurations. The pseudo-inverse, $\mathbf{J}^+$, is given by:

$$\mathbf{J}^+ = \mathbf{J}^T(\mathbf{J}\mathbf{J}^T)^{-1},$$

where $\mathbf{J}$ is the Jacobian matrix.

2. **Quaternion Representation for Orientation:** Using quaternions instead of Euler angles for representing orientation avoids issues like gimbal lock, which is another source of singularity. Quaternions offer a compact and non-redundant representation of rotation while maintaining smooth interpolation between orientations. This approach ensures a more stable orientation control during motion planning and execution.

By combining the pseudo-inverse method and quaternion representation, we effectively resolved singularity-related issues, enabling smooth and precise control of the robotic arm's motion even in challenging configurations.

17.2 Bottlenecks and Mitigations in Simulation

The development of the robotic surgical tool sorting system encountered several challenges during the simulation phase. These bottlenecks and their corresponding mitigations are outlined below:

1. **URDF Exporter Issues:** One of the most significant challenges was the tedious process of using the Unified Robot Description Format (URDF) exporter. The exporter often experienced intermittent crashes

and required manual population of collision and inertial attributes. Additionally, collision tags generated by the SolidWorks URDF exporter defaulted to incorrect values, which complicated interaction modeling.

- *Mitigation:* To address this, the team implemented a systematic approach of saving the progress after adding each frame. Collision and inertial attributes were manually adjusted to ensure accurate representation of the system's dynamics. This manual intervention eliminated interaction inconsistencies and reduced the frequency of crashes.
2. **Translation from CAD to Gazebo:** Another major issue arose during the transfer of designs from CAD software to Gazebo. While the CAD models appeared perfect, the transfer process introduced artifacts, broken assemblies, and disconnected joints in Gazebo. Mates defined in SolidWorks did not translate correctly, further complicating the transition.
 - *Mitigation:* To resolve these issues, redundant components were identified and removed, while necessary parts were merged to simplify the design. This streamlining process reduced the potential for errors during the translation process and ensured functional assemblies in Gazebo.
 3. **STL File Limitations:** Standard STL files only convey geometric information and do not include data about interactions, such as collision properties or inertial values. This limitation hindered accurate interaction modeling.
 - *Mitigation:* Collision and inertial values were manually defined and verified post-export. This step involved generating accurate collision tags and incorporating them into the URDF file, ensuring precise interaction modeling in Gazebo.

These mitigations improved the reliability of simulations and paved the way for a robust robotic system for surgical tool sorting.

17.3 Bottlenecks and Mitigations in Modeling

The modeling phase of the robotic surgical tool sorting system presented several challenges that needed to be addressed for accurate and reliable implementation. The key bottlenecks and their mitigations are as follows:

1. **DH Table Complexity:** The computation of Denavit-Hartenberg (DH) parameters for the robotic arm was complex, requiring the addition of dummy frames with rigid joints to simplify the process.
 - *Mitigation:* Dummy frames were strategically incorporated to ensure consistency in defining coordinate frames and simplify the kinematic chain representation. This approach streamlined the computation of DH parameters and minimized errors in frame alignment.
2. **Singularities in Motion:** Singularities in the robot's motion, such as those affecting the wrist, elbow, and shoulder joints, posed significant challenges. These singularities caused instability, high joint velocities, and difficulty in trajectory planning.
 - *Mitigation:* The team avoided configurations near known singularities during trajectory planning. By carefully defining the workspace and operational constraints, the robot's paths were planned to bypass singular regions. Furthermore, the determinant of the Jacobian matrix was monitored to identify proximity to singularities, ensuring that joint velocities remained within safe limits.
3. **Inverse Kinematics Challenges:** The inverse kinematics problem often lacked unique solutions, especially in scenarios involving redundant joints or complex configurations.
 - *Mitigation:* The Newton-Raphson iterative method was employed, using the pseudo-inverse of the Jacobian matrix to compute changes in joint angles. This approach ensured convergence to feasible solutions while maintaining computational efficiency.
4. **Modeling Tools as Point Masses:** Accurately modeling the dynamics of tools proved challenging due to their varying shapes and masses.
 - *Mitigation:* Tools were approximated as point masses to simplify the dynamic equations. While this approach reduced complexity, it maintained sufficient accuracy for the intended applications.

By addressing these bottlenecks, the modeling process successfully captured the kinematic and dynamic behavior of the robotic system, paving the way for robust simulation and implementation.

17.4 Bottlenecks and Mitigations in Software Implementation

The software implementation of the robotic surgical tool sorting system faced several challenges that required targeted solutions. These bottlenecks and their mitigations are detailed below:

1. **Real-Time Simulation Factor:** Achieving a realistic real-time factor in simulations proved challenging due to computational overheads and resource limitations. Slow simulations impeded iterative testing and validation.
 - *Mitigation:* Optimizations were introduced in the simulation pipeline, such as reducing the simulation complexity and focusing computational resources on critical tasks. This ensured more efficient simulations without compromising accuracy.
2. **Sympy Computation Overhead:** The symbolic computation of Jacobian matrices and other kinematic parameters using Sympy was computationally intensive and time-consuming.
 - *Mitigation:* To address this, a multiprocessing approach was implemented to parallelize the Jacobian computation. By dividing the computations across multiple cores, the process was significantly accelerated, improving overall system responsiveness.
3. **Camera Calibration Errors:** Accurate camera calibration is essential for reliable perception, yet it was prone to errors due to noise and inconsistencies in calibration procedures.
 - *Mitigation:* Robust calibration techniques were employed, including the use of checkerboard patterns and iterative refinement of calibration parameters. This reduced inaccuracies and improved the quality of visual data used for tool detection and localization.
4. **Labeled Dataset for Surgical Tools:** Detecting surgical tools required a labeled dataset, which was both scarce and tedious to generate manually.
 - *Mitigation:* Synthetic datasets were generated using CAD models of surgical tools. By augmenting these datasets with variations in lighting, orientation, and background, the team was able to create a robust dataset suitable for training and testing the detection algorithms.

Through these targeted mitigations, the team overcame the software-related challenges and laid the groundwork for a reliable and efficient robotic system. These improvements enabled more accurate simulations, faster computations, and robust perception capabilities.

18 Lessons Learned

Throughout the course of this project, numerous lessons were learned in the domains of design, simulation, control, and implementation. These insights, drawn from both successes and challenges, encountered during the process

18.1 Key Take Aways

- **Importance of Detailed Planning:** Early in the project, it became evident that detailed planning is crucial for managing complex workflows. The iterative nature of simulation, modeling, and hardware implementation required a well-structured roadmap to ensure progress and avoid redundant efforts.
- **Precision in CAD Modeling:** The process of creating CAD files in Fusion 360 highlighted the importance of precision in modeling. Even small dimensional errors or incorrect joint constraints could propagate significant issues during simulation and deployment, emphasizing the need for attention to detail during the design phase.
- **URDF File Preparation Challenges:** Exporting CAD files into URDF format for simulation in Gazebo proved to be a tedious process. The SolidWorks URDF exporter was prone to crashes, and collision and inertial attributes often needed manual corrections. This highlighted the necessity of understanding file formats and their limitations in robotics projects.
- **Adaptability to Unforeseen Problems:** Issues such as artifacts, broken assemblies, and misaligned joints when transitioning CAD files to Gazebo underlined the importance of adaptability. Simplifying designs by merging components and removing redundant parts was an effective mitigation strategy.

- **Understanding Dynamic Interactions:** Accurate representation of dynamics, such as joint friction and payload effects, required a deeper understanding of kinematics and physics. Simplified assumptions, like point masses and ideal actuators, were helpful during early stages but underscored the need for more advanced modeling in real-world scenarios.
- **Tuning Control Parameters:** The process of tuning PID controllers for the manipulator revealed the complexity of achieving smooth and stable motion. Iterative testing and fine-tuning are essential to achieving desired performance, particularly when working with multi-joint systems.
- **Leveraging ROS for Modular Control:** The integration of ROS topics and services for controlling the EB100 manipulator highlighted the value of modular, scalable communication frameworks. ROS's ability to handle sensor data and control commands made it easier to manage the manipulator's complex operations.
- **Significance of Realistic Simulation:** The simulation environment provided valuable insights into the manipulator's behavior under different scenarios. However, challenges such as achieving a realistic real-time factor emphasized the need for optimizing simulation parameters and reducing computational overhead.
- **Importance of Synthetic Datasets:** The scarcity of labeled datasets for surgical tool detection demonstrated the value of synthetic datasets. Generating datasets from CAD models, combined with augmentation techniques, provided a practical solution to train and validate detection algorithms.
- **Transitioning from Simulation to Hardware:** The ongoing process of hardware implementation, including 3D printing parts, taught valuable lessons about the differences between virtual and physical systems. The alignment between simulated and real-world behaviors will require rigorous testing and further iterations.
- **Collaboration and Teamwork:** Collaboration among team members with varied expertise was critical to addressing challenges effectively. Leveraging diverse skill sets for design, coding, and troubleshooting ensured steady progress and innovation throughout the project.

18.2 Conclusion

The lessons learned during this project reflect the complexity of robotics and the importance of interdisciplinary approaches. From CAD modeling to hardware integration, each phase provided unique challenges and opportunities for growth. These insights will undoubtedly inform future projects, helping us approach similar endeavors with enhanced efficiency, precision, and adaptability.

19 Conclusions

The simulation and development of the EB100 manipulator for surgical tool sorting have provided invaluable insights into both the theoretical and practical aspects of robotic systems. Through meticulous planning and iterative improvements, we successfully simulated the robotic arm's capabilities, addressing critical challenges in modeling, control, and perception.

The simulation phase focused on accurately modeling the manipulator's kinematics, dynamics, and environmental interactions. By leveraging the Unified Robot Description Format (URDF), we integrated the EB100 manipulator into the Gazebo simulation environment, enabling detailed testing of joint motion, trajectory planning, and gripper operations. Key challenges, such as achieving stable joint control through PID tuning, resolving coordinate system misalignments, and optimizing real-time simulation factors, were systematically addressed. The use of ROS for communication and control further enhanced the simulation's functionality, allowing seamless interaction with the robotic arm through well-defined topics and messages.

RViz visualization provided additional clarity by enabling real-time feedback on the manipulator's movements, aiding in the debugging and refinement of control strategies. The gripper's integration into the simulation demonstrated the potential for pick-and-place tasks, laying the groundwork for future improvements in perception and grasping accuracy.

Despite these successes, the simulation process highlighted areas requiring further exploration. The modeling phase revealed challenges in inverse kinematics, singularities, and tool dynamics, which were mitigated using innovative techniques such as the Newton-Raphson method and point mass approximations. Additionally, the software

implementation phase exposed the limitations of computational resources and dataset availability. The generation of synthetic datasets and the adoption of multiprocessing techniques significantly improved performance and robustness.



Figure 22: Components to be assembled

As we transition from simulation to hardware implementation, the focus shifts to assembling the EB100 manipulator and its components. Currently, several parts of the robotic system are undergoing 3D printing, which, upon completion, will enable the full physical realization of the manipulator. This phase will involve extensive testing to ensure that the hardware accurately reflects the simulated model and performs reliably in real-world conditions.

In conclusion, the EB100 manipulator simulation has been a pivotal step toward achieving an efficient surgical tool sorting system. While the simulation process provided a strong foundation for understanding and optimizing the manipulator’s capabilities, the forthcoming hardware implementation will allow us to validate these findings and further refine the system’s design and functionality. This iterative approach ensures that the final robotic solution is both robust and effective in meeting the demands of modern healthcare environments.

20 Future Work

The following areas of improvement and extension have been identified to enhance the performance and functionality of the robotic system for surgical tool sorting:

1. **Consider Dynamics:** Future work should incorporate dynamic modeling of the system to account for real-world scenarios. This includes:
 - *Modeling the effect of tool handling on joints:* When the robot picks up tools, the added weight and distribution of the load can influence the joint dynamics, requiring precise torque calculations to maintain stability and accuracy.

- *Joint frictions:* Frictional forces in the joints can impact the smoothness of motion and energy efficiency. Incorporating these forces into the model would improve the accuracy of predictions and system performance.
 - *Optimal gripper force:* Developing algorithms to calculate the optimal force for grasping tools of varying weights and shapes ensures reliable handling without causing damage to the tools or gripper.
2. **Sensing on Grippers:** Enhancing the gripper with force sensors on the fingers or modeling the resistance of the gripper servo would allow the system to sense the applied force during operations. This feedback could improve handling precision, particularly when dealing with delicate or irregularly shaped tools.
 3. **Fully Working Hardware Implementation:** While the current system is partially simulated, a complete hardware implementation is essential for testing the system in real-world scenarios. This involves assembling all components, integrating control systems, and ensuring operational reliability under varied conditions.
 4. **Signal Filtering and Noise Reduction:** Measured signals from sensors often include noise, which can affect torque oscillations and lead to performance instability. Applying advanced filtering techniques, such as Kalman filters or low-pass filters, would improve the accuracy of sensor data and enhance system robustness.

These areas represent critical steps in transitioning from the conceptual and simulated stages to a fully operational robotic system. Addressing these challenges will enable the development of a highly reliable and efficient robotic solution for surgical tool sorting in sterile environments.

21 Links to GitHub Repository and demonstration videos

- **GitHub Repository:** Repository

References

- [1] R. Song *et al.*, “Fast surgical instruments identification system to facilitate robot picking,” in *2022 IEEE/ASME International Conference on Advanced Intelligent Mechatronics (AIM)*, 2022, pp. 838–843.
- [2] Y. Xu *et al.*, “Robotic handling of surgical instruments in a cluttered tray,” *IEEE Transactions on Automation Science and Engineering*, vol. 12, no. 2, pp. 775–780, 2015.
- [3] Q. Wu *et al.*, “Coordinated control of a dual-arm robot for surgical instrument sorting tasks,” *Robotics and Autonomous Systems*, vol. 112, pp. 1–12, 2019.
- [4] U. Robots, “Dh parameters,” *IEEE Transactions on Automation Science and Engineering*, 2015.

22 Appendix

A Project Components

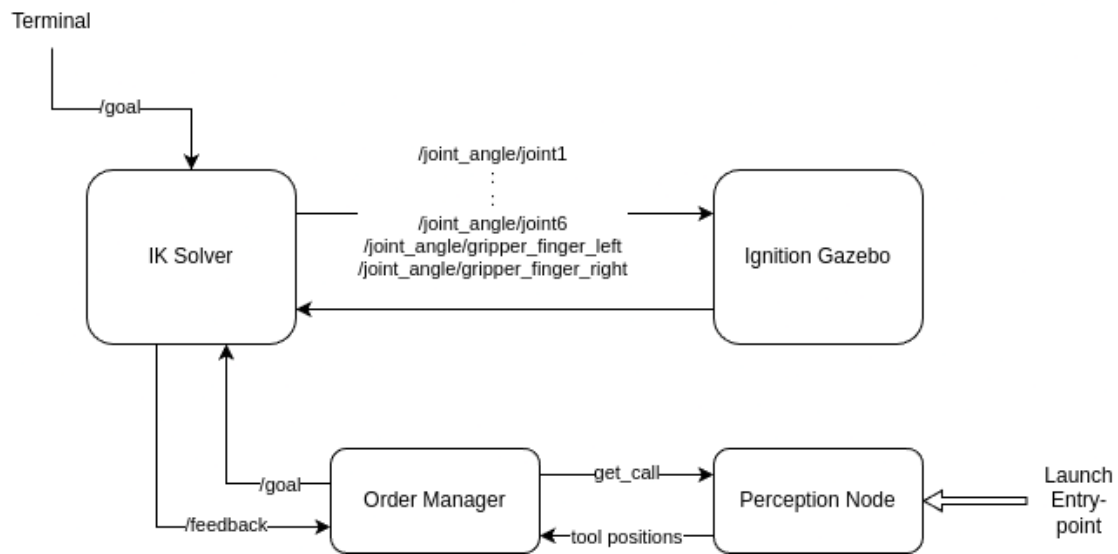


Figure 23: Project Components

B Workflow

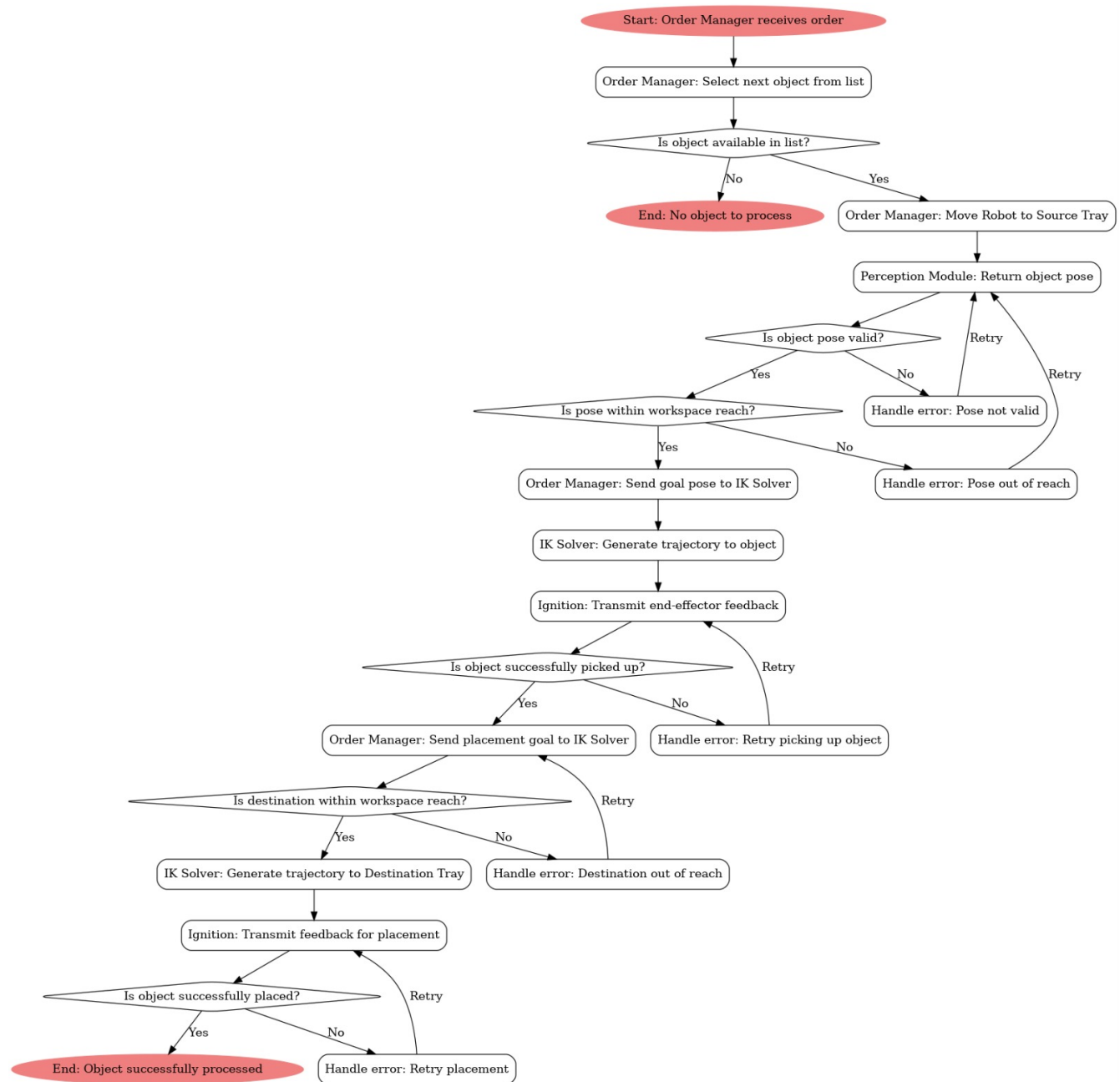


Figure 24: Project workflow